open-media-drc

Digital Room Correction media chain for Linux and FreeBSD — User and Administrator Manual

Generated from the repository Markdown documentation

v0.91.0-24-g53bc081-dirty (2026-08-17)

Contents

1 Introduction **3**
- 1.1 Repository map . . . 5

2 Components **6**
- 2.1 upmpdcli, libupnpp, libnpupnp (UPnP/OpenHome front-end) . . . 6
- 2.2 MPD — Music Player Daemon . . . 6
- 2.3 The loopback: snd-aloop (Linux) / virtual_oss (FreeBSD) . . . 6
- 2.4 BruteFIR (delleceste fork) . . . 7
- 2.5 open-media-drc proper: drc.sh and the configuration tree . . . 7
- 2.6 omdrc-ctrl — web control panel . . . 7
- 2.7 video/ — mpv playback and the phone web remote . . . 7
- 2.8 browser-nodrc — DRC bypass for browsers . . . 7

3 Installation **8**
- 3.1 Dependencies . . . 8
- 3.2 Build and install order . . . 8
- 3.3 Files that must live in /etc . . . 10
- 3.4 Linux specifics . . . 11
  - 3.4.1 The MPD `User=` drop-in caveat (Arch) . . . 11
  - 3.4.2 USB DAC hotplug (udev + systemd) . . . 11
- 3.5 FreeBSD specifics . . . 11
  - 3.5.1 Why both a boot probe and a hotplug edge . . . 12

4 Usage: drc.sh, filters, and configuration **13**
- 4.1 drc.sh — the single control point . . . 13
- 4.2 MPD outputs . . . 14
- 4.3 The filters/ and configs/ trees . . . 15
- 4.4 Filter generation workflow . . . 15
- 4.5 Browser audio without DRC . . . 17

5 Filter provenance and verification **18**
- 5.1 The three repositories . . . 19
- 5.2 Geometry, design, variant . . . 20
- 5.3 The bundle . . . 20
- 5.4 What is hashed . . . 21
- 5.5 The scripts . . . 21
- 5.6 The workflow . . . 22

5.7 Verification at install time . . . 23
5.8 Verification at runtime . . . 23
5.9 What deliberately cannot go green . . . 24

6 Tools **25**
6.1 omdrc-ctrl — the web control panel . . . 25
6.2 Video: mpv playback + phone web remote . . . 27
6.2.1 Playback launchers . . . 27
6.2.2 The web remote (`video/webremote/`) . . . 28
6.3 Glitch detection . . . 28
6.4 Bit-perfect verification . . . 29
6.5 scripts/ — helper tools . . . 29

7 FreeBSD peculiarities **31**
7.1 Naming and packages . . . 31
7.2 OSS instead of ALSA; virtual_oss and cuse . . . 31
7.3 Known FreeBSD issues and their status . . . 31
7.4 Video-related FreeBSD constraints . . . 32
7.5 Toward a real FreeBSD port . . . 32

8 Appendix A — FreeBSD patches in detail **33**
8.1 uaudio(4) patches (`freebsd-uaudio-patch/`) . . . 33
8.1.1 1. `uaudio-clock-before-alt.c.patch` — rate-change cold-open silence . . . 33
8.1.2 2. `uaudio-shared-clock-fix.c.patch` — the 44.1 kHz flicker (bug #295933) 34
8.1.3 3. `uaudio-feedback-follow.c.patch` — candidate (unbuilt) . . . 34
8.1.4 Applying and building . . . 35
8.2 virtual_oss / cuse patches (`freebsd-virtual-oss-patch/`) . . . 35
8.2.1 1. Runtime livelock: `virtual_oss-settrigger-sync-deadlock.patch` . . . 35
8.2.2 2. Teardown deadlock: userland device destroy + kernel refleak . . . 36
8.2.3 Applying and building . . . 36
8.3 Kodi OSS sink patch (`kodi-virtual-oss-patch/`) . . . 37

9 Appendix B — The FreeBSD port plan **38**
9.1 Why the repo cannot be ported as-is . . . 38
9.2 Phase 0 — Upstream the out-of-tree pieces (prerequisite, in flight) . . . 39
9.3 Phase 1 — Make the repo package-friendly . . . 39
9.4 Phase 2 — The port itself . . . 40
9.5 Phase 3 — Submission and maintenance . . . 40
9.6 Recommended order of value . . . 40

10 Appendix C — Bit-perfect test assets and cross-OS comparison **41**
10.1 Test assets (`tests/`) . . . 41
10.2 Other rates and sample widths . . . 41
10.3 Verification status . . . 42
10.4 Cross-OS byte comparison . . . 43

11 Appendix D — Source document index **45**
11.1 Keeping this manual up to date . . . 46

Chapter 1

Introduction

open-media-drc is the complete software stack of a headless, high-quality music and video playback appliance with **Digital Room Correction (DRC)**. It runs on both **Linux** (reference: Arch) and **FreeBSD** (reference: 15.1 on an Intel NUC), driving an OKTO RESEARCH DAC8 STEREO USB DAC.

The core idea: audio from any source — UPnP/OpenHome streaming (Qobuz), local files, Blu-ray discs, network streams — is routed through **BruteFIR**, a fast FIR convolution engine, which applies room-correction filters designed with Room EQ Wizard (REW) and DRC tooling, before reaching the DAC. When correction is off, the chain collapses to a verified **bit-perfect** direct path.

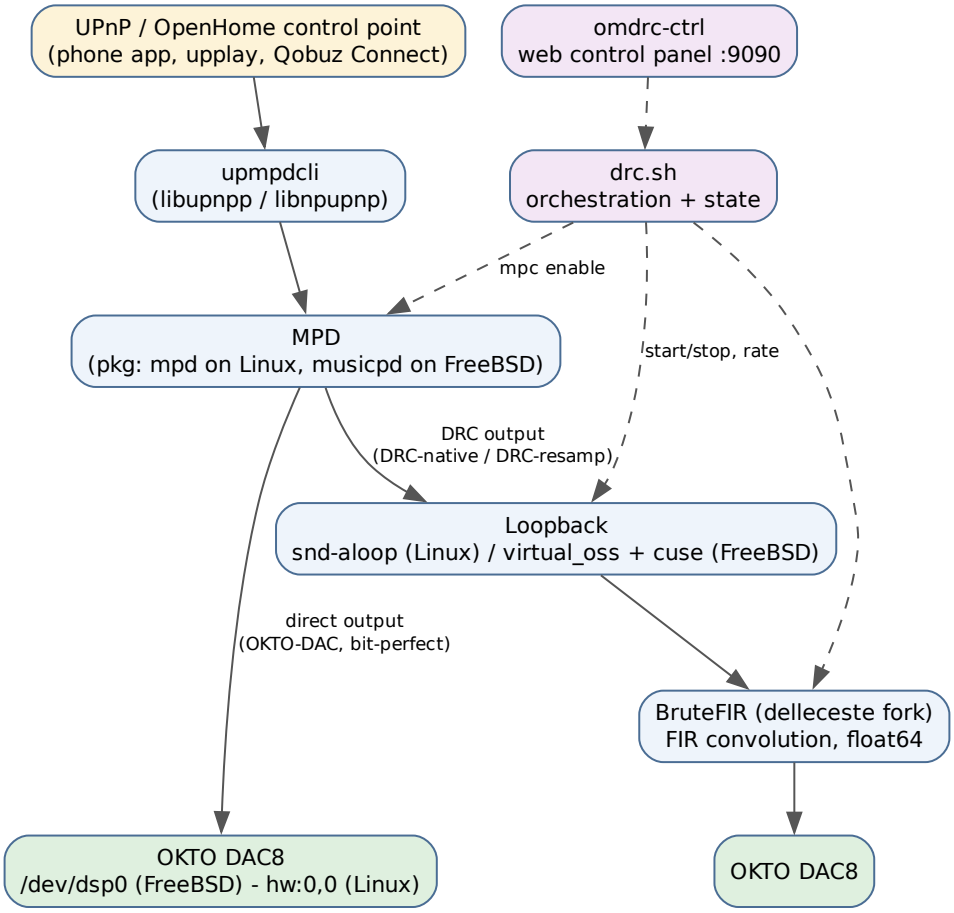

Figure 1.1: The full audio playback chain, from control point to DAC. Solid arrows carry audio; dashed arrows are control.

The design principles that shape everything in the repository:

- **The DAC's presence is the single condition that drives DRC.** DAC present means DRC up, replaying the last saved rate and variant; DAC absent means DRC down, MPD playing straight to the direct output. Boot, hotplug, and shutdown all funnel into that one rule.
- **Only two resting states exist**: DRC fully up (BruteFIR processing), or direct output. A failed start rolls back — there is no resting state where the loopback runs without BruteFIR.
- **Run-from-repo.** The live configuration files and scripts are symlinks into the git checkout wherever possible; `git pull` is the whole update path. Only "deploy glue" parsed at early boot (systemd units, udev rules, rc.d scripts, devd rules) is *copied* into system paths, because those are parsed before a separately mounted `/home` is available.
- **Verify, don't assume.** The repository ships tooling to *prove* the chain is bit-perfect (a USB wire tap), to detect and classify glitches, and to monitor the whole chain from a phone.

1.1 Repository map

Path	Contents
`drc.sh`, `drc-status.sh`	DRC orchestration and status
`configs/<geometry>/`	Per-rate BruteFIR configurations
`filters/<geometry>/<rate>/`	Raw FIR filter coefficients (FLOAT64_LE)
`mpd/`	MPD configuration templates (`mpd.conf.in` Linux, `musicpd.conf.in` FreeBSD)
`etc/`	Service glue: `systemd/`, `modules-load.d/` (Linux); `rc.d/`, `devd/` (FreeBSD)
`omdrc-ctrl/`	Web control panel (Flask)
`video/`	mpv playback launchers + phone web remote
`browser-nodrc/`	Browser launchers that temporarily bypass DRC
`scripts/`	Filter conversion, headroom, verification helpers
`tests/`	Bit-perfect test signal
`freebsd-uaudio-patch/`	FreeBSD kernel `uaudio(4)` patches (Appendix A)
`freebsd-virtual-oss-patch/`	FreeBSD `virtual_oss` / `cuse` patches (Appendix A)
`kodi-virtual-oss-patch/`	Kodi OSS-sink enumeration patch (Appendix A)
`doc/`	Measurement plots, verification and glitch docs, this manual

Chapter 2

Components

The playback chain is assembled from the following components, in signal order.

2.1 upmpdcli, libupnpp, libnpupnp (UPnP/OpenHome front-end)

`upmpdcli` turns MPD into a UPnP/OpenHome media renderer, so any control point (a phone app, `upplay`, Qobuz Connect) can drive playback. It is built from source, bottom-up, as three standard meson projects: **libnpupnp** (UPnP base library; needs libcurl, libmicrohttpd, expat), **libupnpp** (C++ wrapper), then **upmpdcli** itself (needs jsoncpp, libmpdclient). The optional Qobuz plugin needs python3 with `requests`. Prebuilt packages exist (FreeBSD port `upmpdcli`, Arch AUR); source builds are used here to track upstream.

2.2 MPD — Music Player Daemon

The actual player. Installed **from the OS package**:

> **Naming: MPD is packaged as `mpd` on Linux, but as `musicpd` on FreeBSD.** The FreeBSD package/port is `audio/musicpd`, the daemon binary is `musicpd`, the service is `service musicpd ...`, and the bundled command-line client is `musicpc` (aliasing `mpc`). Everywhere this manual says "MPD", read `mpd` on Linux and `musicpd` on FreeBSD.

MPD must have the **soxr** resampler and the ALSA (Linux) / OSS (FreeBSD) output plugins enabled — both stock packages do. MPD is configured with three outputs (section 4.2): the direct DAC output and two DRC loopback outputs.

2.3 The loopback: snd-aloop (Linux) / virtual_oss (FreeBSD)

BruteFIR needs to receive the player's audio. That is done with a loopback device MPD plays into and BruteFIR reads from:

- **Linux**: the `snd-aloop` ALSA kernel module (loaded via `etc/modules-load.d/`).
- **FreeBSD**: `virtual_oss`, a userland OSS mixing/routing daemon from the base system, which creates character devices through the `cuse(3)` kernel facility. `drc.sh` starts it per rate with a play node (`/dev/dsp.play`, where MPD writes) and a synchronized loopback node

(`/dev/dsp.loop`, where BruteFIR reads): the `-L` loopback. The `cuse` kernel module must be loaded.

2.4 BruteFIR (delleceste fork)

The convolution engine, built from **`github.com/delleceste/brutefir`** — a fork of Anders Torger's classic BruteFIR adding FreeBSD OSS fixes (`bfio_oss` fragment-size fix, `brutefir_loopback -L` fix, and a passthrough-config default). It processes audio entirely in **float64**, convolving the per-rate `L.raw`/`R.raw` FIR filters and writing to the DAC. Requires FFTW3 in both single and double precision; ALSA on Linux (OSS support is built in on FreeBSD). Built with CMake; modules install to `/usr/local/lib/brutefir`.

2.5 open-media-drc proper: drc.sh and the configuration tree

`drc.sh` is the single control point of the DRC pipeline: it starts and stops BruteFIR and the loopback, selects the MPD output, primes the DAC on rate changes, serializes concurrent runs, records persistent state, and rolls back to direct output on failure. Section 4 covers it in full.

2.6 omdrc-ctrl — web control panel

A lightweight Flask web app serving a mobile-friendly, dark, touch-first control panel on the LAN (default port 9090). It exposes DRC control buttons, full audio-chain health monitoring with a bit-perfect verdict, a live spectrum analyzer, and DRC filter-response charts. Section 6.1.

2.7 video/ — mpv playback and the phone web remote

Blu-ray discs, DVDs, files and streams played through mpv with audio routed through the same DRC chain, controlled either from KDE Connect (MPRIS) or from a dedicated phone web remote (the mpv-only Kodi replacement). Section 6.2.

2.8 browser-nodrc — DRC bypass for browsers

Web browsers cannot easily route into the DRC loopback, and BruteFIR holds the DAC single-open while DRC runs. `browser-nodrc/` provides one launcher per browser (Firefox, Chromium, Chrome) that snapshots the DRC state, runs `drc.sh off`, launches the browser in the foreground, and restores the exact pre-launch state from an EXIT trap — even if the browser crashes.

Chapter 3

Installation

3.1 Dependencies

Build tools for all from-source components: a C/C++ compiler, **meson + ninja** (upmpdcli stack), **cmake** (BruteFIR fork, omdrc-ctrl), **pkg-config**, and git.

Component	Runtime deps	FreeBSD pkg	Arch pacman
libnpupnp 6.3.0	libcurl, libmicrohttpd, expat	`curl libmicrohttpd expat2`	`curl libmicrohttpd expat`
libupnpp 1.0.4	libnpupnp + the above	(above)	(above)
upmpdcli 1.9.17	libupnpp, jsoncpp, libmpdclient	`jsoncpp libmpdclient`	`jsoncpp libmpdclient`
upmpdcli Qobuz plugin	python3 + requests	`python3 py311-requests`	`python python-requests`
MPD	soxr resampler + ALSA/OSS output	**`musicpd`**	**`mpd`**
BruteFIR (fork)	FFTW3 single+double; ALSA on Linux	`fftw3 fftw3-float`	`fftw alsa-lib`
Loopback	—	`virtual_oss` (+ `cuse`)	`snd-aloop` kernel module
omdrc-ctrl	python3, flask>=2.3, markdown>=3.5, numpy>=1.21 (optional)	`py311-flask py311-Markdown py311-numpy`	`python-flask python-markdown python-numpy`

3.2 Build and install order

1. The upmpdcli stack (bottom-up; each a standard meson project):

```
for p in libnpupnp-6.3.0 libupnpp-1.0.4 upmpdcli-1.9.17; do
  cd ~/Downloads/$p
  meson setup build --prefix=/usr/local
  ninja -C build
```

```
  sudo ninja -C build install
done
sudo ldconfig 2>/dev/null || true   # Linux: refresh the linker cache
```

2. MPD — from the OS package:

```
sudo pkg install musicpd      # FreeBSD
sudo pacman -S mpd            # Arch Linux
```

3. BruteFIR — from the fork:

```
git clone https://github.com/delleceste/brutefir ~/Downloads/brutefir
cd ~/Downloads/brutefir
cmake -B build && cmake --build build
sudo cmake --install build    # modules -> /usr/local/lib/brutefir
```

4. open-media-drc (DRC engine + web UIs + DAC hotplug) — the classical CMake build:

```
git clone --recursive https://github.com/delleceste/open-media-drc
↪  ~/DRC/open-media-drc
cd ~/DRC/open-media-drc
cp host.cmake.sample host.cmake   # AUDIO_USER (defaults to the invoking user),
$EDITOR host.cmake                #   GEOMETRY, GEOMETRIES, OMDRC_SITE_DATA_DIRS,
                                  #   MUSIC_DIR, VIDEO_DIR, OMDB_API_KEY
mkdir build && cd build
cmake .. -C ../host.cmake
make
sudo make install                 # -> $PREFIX (default /usr/local)
```

`host.cmake` is read only by `-C`, and only for cache entries that do not exist yet. A plain `cmake ..` configures the whole project from the built-in defaults, and adding `-C` afterwards cannot repair that build directory — CMake skips an initial-cache assignment whose entry is already set. The symptom is silent: everything configures, with the wrong values. The project therefore detects it, because `host.cmake` sets a marker the check looks for:

```
CMake Warning at CMakeLists.txt:50 (message):
  host.cmake exists but this build directory was not initialised from it, so
  the built-in defaults are in effect.  Adding -C now will not help; start a
  fresh build directory:

      rm -rf build && mkdir build && cd build && cmake -C ../host.cmake ..
```

A `host.cmake` copied from a version of the sample that predates the marker will trip this warning; add the one line the sample carries near the top.

`host.cmake` (the successor to the old `config.env`) is the single source of box-specific values; CMake renders every config from it and installs the DRC engine (`drc.sh` behind the `omdrc` / `omdrc-status` wrappers), the site data (brutefir configs + impulse-response filters for `GEOMETRY` and every set listed in `GEOMETRIES`, each looked up along `OMDRC_SITE_DATA_DIRS` and reported at configure time), both web UIs (omdrcctrl :9090 as a **system** service running as the audio user; omdrcvideo :9080 as a

`--user` service, since it drives the desktop-session mpv), and the DAC-hotplug glue. The install prints the OS-specific enable steps and the one or two files that must be copied into `/etc` (next section).

The older `./install.sh` rendered the `*.in` templates in place and ran everything straight from the checkout (`git pull` = update). That run-from-repo mode still works but is superseded by the CMake install, which now covers the whole DRC stack — engine, DAC-hotplug glue, both web UIs, and the MPD + upmpdcli renderer configs/units. `install.sh` remains only for the desktop glue not yet in CMake: the `browser-nodrc .desktop` launcher entries and the Linux `snd-aloop` module-load. (The video mpv-idle autostart entry moved to CMake: `make install` puts it under `$PREFIX/share/omdrcvideo/autostart/` on both OSes, `make user-install` links it into `~/.config/autostart/`.)

5. BruteFIR defaults — BruteFIR reads float precision, partition size and the I/O devices from `~/.config/BruteFIR/brutefir_defaults.conf`. The per-rate configs deliberately leave their `input`/`output` blocks empty and inherit devices from this file, so it **must** be deployed:

```
mkdir -p ~/.config/BruteFIR
cp brutefir_defaults.linux.conf ~/.config/BruteFIR/brutefir_defaults.conf  #
↪  Linux/ALSA
cp brutefir_defaults.conf       ~/.config/BruteFIR/brutefir_defaults.conf  #
↪  FreeBSD/OSS
```

If this file is missing, BruteFIR (>= 1.1) silently auto-generates a broken fallback `~/.brutefir_defaults`, every start fails with *"Parse error: path not set ... module 'file'"*, and `drc.sh` rolls back to direct output — so DRC never comes up at boot.

3.3 Files that must live in /etc

The CMake install copies everything into `$PREFIX` (default `/usr/local`); the update path is `git pull` followed by `cmake --build build && sudo cmake --install build`, not an in-place edit of the checkout. Two files are read *before* `$PREFIX` is in the relevant search path, so they are handled specially:

- **Linux udev rule** — udev only scans `/etc/udev/rules.d` and `/usr/lib/udev`, never `/usr/local/lib/udev`. The install places `99-usb-audio-drc.rules` under `$PREFIX/lib/udev/rules.d` and prints the one-line copy into `/etc/udev/rules.d` (then `udevadm control --reload`).
- **MPD `/etc` drop-in** — overriding the distro `mpd` unit's `User=mpd` requires a real file in `/etc/systemd/system/mpd.service.d/` (a drop-in in `/etc` beats one in `/usr/lib`; see the caveat below).

Everything else stays under `$PREFIX` and needs no `/etc` copy: the systemd units (`$PREFIX/lib/systemd/{system,us`
which systemd *does* scan), the FreeBSD rc.d scripts and devd rule (`$PREFIX/etc/{rc.d,devd}`, scanned natively), `drc.sh`, the filters and configs. The BruteFIR defaults still go to `~/.config/BruteFIR/` (below).

3.4 Linux specifics

3.4.1 The MPD `User=` drop-in caveat (Arch)

The Arch `mpd` package ships a systemd drop-in (`/usr/lib/systemd/system/mpd.service.d/00-arch.conf`) setting `User=mpd`. Drop-ins always apply *on top of* the main unit, so a full unit override at `/etc/systemd/system/mpd.service` **cannot** override that `User=` — it silently loses. The repo therefore ships a **counter-drop-in** (`etc/systemd/system/mpd.service.d/open-media-drc.conf`, rendered by `install.sh`) that sets `User=@AUDIO_USER@` and the repo config path; a drop-in in `/etc` beats one in `/usr/lib`. It must be a **real file copied into `/etc`, not a symlink** (early-boot parse, see above). Do not create a full `/etc/systemd/system/mpd.service`.

3.4.2 USB DAC hotplug (udev + systemd)

File	Installed to	Purpose
`99-usb-audio-drc.rules`	`/etc/udev/rules.d/`	Triggers the service on DAC plug/unplug
`etc/systemd/system/drc-usb-audio.service`	`/etc/systemd/system/`	Starts/stops DRC
`mpd.service.d/open-media-drc.conf`	`/etc/systemd/system/mpd.service.d/`	MPD user/config drop-in

The udev rule matches any USB sound-card control device and pulls in `drc-usb-audio.service` (`Type=oneshot`, `RemainAfterExit=yes` so the several `controlC*` events of one plug never start duplicate BruteFIR instances). `ExecStart` is `drc.sh restore` after a 1 s settle; `ExecStop` is `drc.sh stop`. Because udev synthesizes ADD events for already-present devices at boot, the same service covers boot and hotplug.

Manual control: `sudo systemctl start|stop drc-usb-audio.service`, `journalctl -fu drc-usb-audio.service`.

3.5 FreeBSD specifics

The CMake install (step 4) renders the FreeBSD service glue from the port's templates and places it under `$PREFIX`; devd and rc both scan those paths natively, so there is no `/etc` copy step:

File	Installed to	Purpose
`etc/rc.d/brutefir_drc`	`$PREFIX/etc/rc.d/`	Worker: runs `omdrc restore` / `stop` as the audio user
`etc/rc.d/drc_usb_audio`	`$PREFIX/etc/rc.d/`	Entry point: probe at boot, devd target
`etc/devd/usb-audio-drc.conf`	`$PREFIX/etc/devd/`	Attach/detach triggers

Enable **only** the entry point in `/etc/rc.conf`:

```
drc_usb_audio_enable="YES"
# optional overrides:
brutefir_drc_drcsh="/home/user/DRC/open-media-drc/drc.sh"
drc_usb_audio_start_delay="1"
```

`brutefir_drc` is the worker invoked by `drc_usb_audio`; it must stay installed but **unenabled**, otherwise boot starts the chain twice. The devd rule calls `service ... onestart/onestop` directly, so hotplug needs no rc.conf flags. `drc.sh` serializes mutating runs under a lock (`lockf` on FreeBSD, `flock` on Linux), so overlapping triggers are safe.

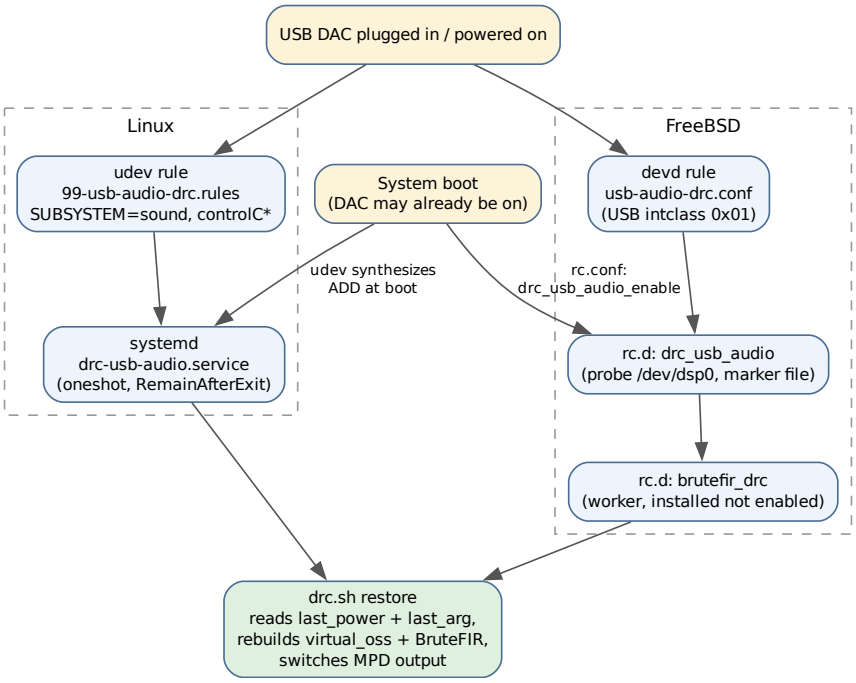

Figure 3.1: Boot and hotplug event chains on both platforms. Both funnel into the single `drc.sh restore` path.

3.5.1 Why both a boot probe and a hotplug edge

Hotplug events only cover the "switched on later" case. On FreeBSD, devd does not reliably receive an attach that happened before it opened `/dev/devctl`, so a DAC already on at boot would never produce an actionable event. Therefore `drc_usb_audio`'s start does a *level* check — sleep the settle delay, probe for `/dev/dsp0`, act only if present — while devd provides the *edge* trigger for later attaches. A tmpfs marker file (`/var/run/drc_usb_audio.active`) makes repeated triggers idempotent.

Chapter 4

Usage: drc.sh, filters, and configuration

4.1 drc.sh — the single control point

`drc.sh <rate>|resamp|restore|off|stop [variant]`

Verb	Effect
`<rate>`	Start BruteFIR at 44100 / 48000 / 88200 / 96000 / 192000 Hz; restart the loopback at the same rate; switch MPD to `DRC-native`
`resamp`	Everything at 192 kHz; switch MPD to `DRC-resamp` (MPD resamples with soxr)
`restore`	Re-apply the state at shutdown: stays off if `last_power` is `off`, else replays `last_arg` (falls back to 192000)
`off`	Stop BruteFIR + loopback; MPD back to direct output; **records the off state**. The user-facing disable
`stop`	Same teardown as `off` but does **not** record off. Used by service stop paths so a reboot of a *running* system is restored
`variant`	Optional second argument (a config-filename suffix): selects an alternate filter set; superseded by `design`

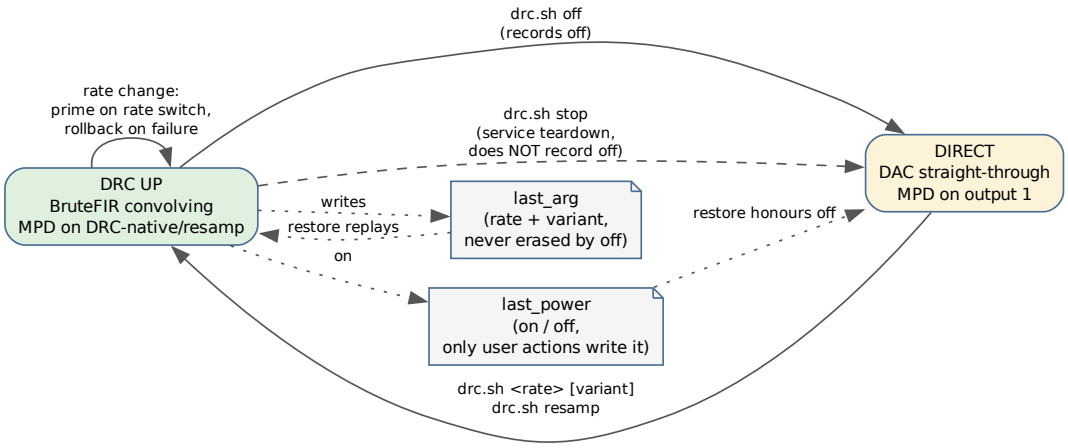

Figure 4.1: drc.sh verbs and the two persistent state files.

State lives in two files in the repo root (git-ignored), so the on/off state and the remembered rate stay independent:

- `last_arg` — the last *active* rate and optional variant (`192000`, `resamp`, `192000 <variant>`). Written on each successful run, **never erased by `off`** — turning DRC back on restores the last rate. It records the *desired* state: a failed start never rewrites it, so the next trigger retries the same configuration.
- `last_power` — `on` or `off`. Only real user actions write it (a rate run writes `on`, `off` writes `off`; `stop` leaves it alone).

The speaker geometry is not an argument: it is the `GEOMETRY/POSITION` variable at the top of `drc.sh` (currently `120.blue`).

What a (re)start does, in order: stop any running BruteFIR and wait for the DAC to be released; disable all MPD outputs; (FreeBSD) restart `virtual_oss` at the target rate and wait for `/dev/dsp.loop`; prime the DAC if the rate changed; start BruteFIR and **verify it stays up**; enable the matching MPD output; record the state. If BruteFIR cannot come up, `drc.sh` **rolls back** — stops the loopback and re-enables the direct output — leaving a clean, audible system equivalent to `off`.

The priming quirk: the OKTO DAC routes silence on the *first* stream opened at a new sample rate; a second open fixes it. On a detected rate change `drc.sh` opens BruteFIR once, tears it down, then starts it for real. (The kernel-level fix for this is the clock-before-alt patch, Appendix 8.1; with it installed, `DAC_PRIME_CYCLES` defaults to 0.)

`drc.sh status` (and `drc-status.sh`) reports the *actual, observed* state — config, loopback/ALSA rate, BruteFIR, MPD output and rate — derived from what is running, not from `last_arg`.

4.2 MPD outputs

Output	Device	`format`	Meaning
`OKTO-DAC`	direct DAC	—	bit-perfect direct path, no DRC
`DRC-native`	loopback	`*:*:*`	native-rate DRC; MPD does **not** convert; you pick the `drc.sh` rate matching the track
`DRC-resamp`	loopback	`192000:24:2`	MPD resamples everything to 192 kHz (soxr); for mixed-rate playlists

`*:*:*` (rate:bits:channels, asterisk = unenforced) is deliberate: native DRC mode must not force MPD conversion. `drc.sh 192000` and `drc.sh resamp` both use the 192 kHz BruteFIR config but are distinct modes: the web UI shows them as *Flat 192 kHz* vs *Flat auto-resample*. In native mode the sample rate is the routing selector, never the bit depth — BruteFIR processes in floating point.

4.3 The filters/ and configs/ trees

```
filters/<geometry>/<rate>/{L,R}.raw          raw FLOAT64_LE FIR coefficients
filters/<geometry>/<rate>/@<design>/{L,R}.raw   an immutable A/B design
filters/<geometry>/provenance/<design>.json  hash-bound manifest
filters/<geometry>/analysis/<design>.json    precomputed response traces
filters/<geometry>/rew/                      REW-exported source WAVs
configs/<geometry>/brutefir-<rate>[@<design>].conf.in
```

These two trees are the *site data*. They need not live in this checkout: CMake resolves them along `OMDRC_SITE_DATA_DIRS` and the design scripts along `OMDRC_SITE_ROOT`, so one room's measurements can be a separate repository while the engine ships only the generic `flat` set. See Filter provenance and verification.

`drc.sh` builds the config path as `configs/<geometry>/brutefir-<actual_rate><variant>.conf` (for `resamp`, `actual_rate` is 192000) and BruteFIR reads the filter paths from that config — the config is the authoritative link. A variant works only if the matching config file exists; nothing is auto-discovered.

For a new rate or variant, create all the pieces: the `L.raw`/`R.raw` pair, the config pointing at them, and verify the `attenuation:` (below).

4.4 Filter generation workflow

This is the low-level route: it produces coefficients but no provenance bundle, so the web UI cannot verify the result and will not show stored measurements beside it. For deployable work use the audited workflow in Filter provenance and verification; what follows is still useful for experiments and is what the audited path drives underneath.

1. Export the corrected impulse responses from REW as WAV (typically 48 kHz).
2. Convert for every rate directory:

```
scripts/REW2raw-all-rates.sh \
  -L filters/120.blue/rew/FLX-trimmed-48k.wav \
  -R filters/120.blue/rew/FRX-trimmed-48k.wav \
  -o filters/120.blue
```

`REW2raw.sh` (called per rate) resamples with SoX at very high quality (`-v -L -s`, 64-bit float intermediate) and applies **one deterministic FIR coefficient scale** — `scale = Fs_source / Fs_target` (Julius O. Smith, *Physical Audio Signal Processing*) — never peak normalisation, which would alter the intended filter gain. A `sox.txt` log records every command and measured stat.

3. Compute the clipping headroom and set `attenuation:` in each config:

```
python3 scripts/headroom_calc.py
```

BruteFIR works in float64, so clipping can only happen at the output boundary where the filter has gain > 0 dB. The script FFTs each raw filter, takes the worst-case gain across frequency, adds a safety margin (default 1 dB), and reports the suggested per-pair attenuation (BruteFIR applies one `attenuation:` per coeff block to both channels, so the louder channel limits). Attenuation in float64 is lossless — the only goal is avoiding clipping.

The measured result of the current filter set (`120.blue`, v1.5.0):

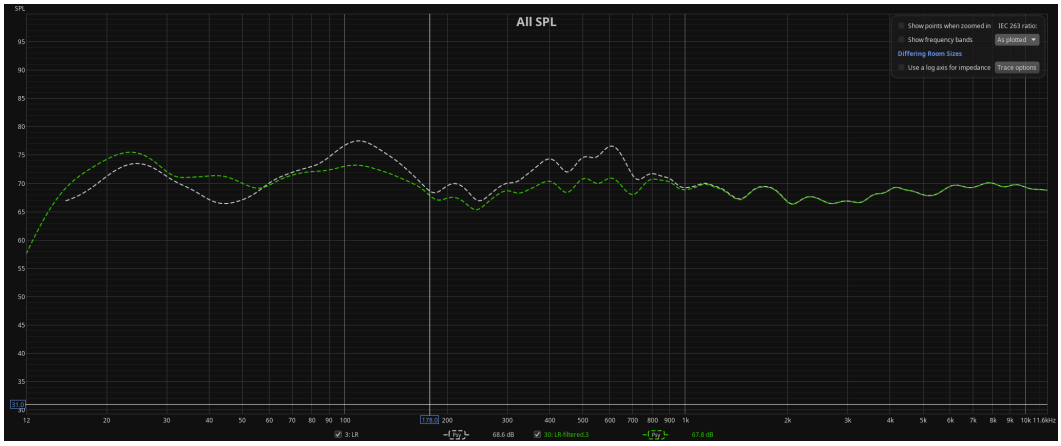

Figure 4.2: Amplitude response: corrected vs uncorrected.

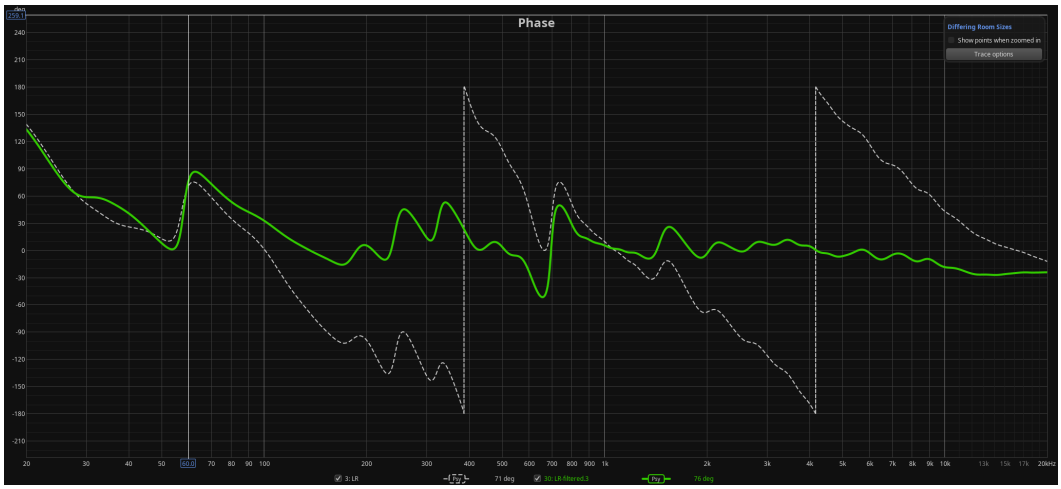

Figure 4.3: Phase response: corrected vs uncorrected.

4.5 Browser audio without DRC

While DRC runs, BruteFIR holds the DAC single-open and browsers get silence. Each `browser-nodrc/` launcher: (1) snapshots `last_power` + `last_arg`; (2) runs `drc.sh off`; (3) runs the browser in the foreground; (4) restores the exact pre-launch state from an `EXIT/INT/TERM` trap. It deliberately does *not* use `drc.sh restore` (which would honour the `off` just recorded and leave DRC down). Firefox runs with `--no-remote`; Chrome/Chromium detect an already-running instance with `pgrep` and then just hand over the URL without touching DRC. The rendered `.desktop` launchers are symlinked into `~/.local/share/applications/`.

Chapter 5

Filter provenance and verification

A room-correction filter is an empirical artifact: it is only as good as the measurement session it came from, and it is indistinguishable, once converted to a `.raw` blob of float64 coefficients, from any other blob of the same length. The response page in the web UI shows *stored* room measurements — curves that were computed offline, months earlier, from the original REW exports. Showing them beside a filter that is not the one they describe would be worse than showing nothing, because it looks authoritative.

This chapter describes the machinery that prevents that: a chain of content hashes running from the REW exports in the source repository to the coefficient bytes BruteFIR has actually loaded, with a refusal at every link that cannot be checked.

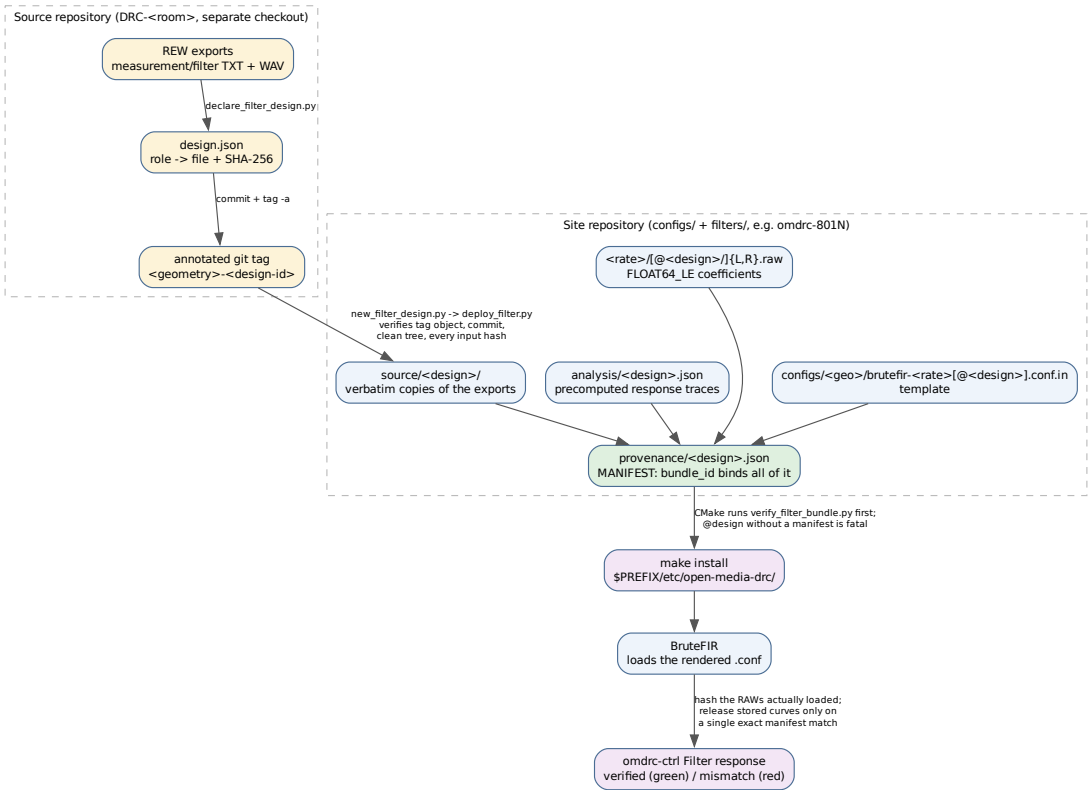

Figure 5.1: Provenance chain: each labelled arrow is a content check that must pass before a design is deployable, installable, or shown as verified.

5.1 The three repositories

The workflow spans three trees, deliberately kept apart:

Tree	Holds	Example
Source repository	REW projects and exports, the role declaration, annotated tags	`../DRC/DRC-120.blue`
Site repository	`configs/<geometry>/`, `filters/<geometry>/` — one physical room	`omdrc-801N`
Engine repository	`drc.sh`, the scripts, CMake, and the generic `flat` set	`open-media-drc`

The site data used to live inside the engine checkout. It no longer has to: `configs/<geo>` and `filters/<geo>` are resolved through a *site root*, so personal room measurements can be versioned and deployed independently of the software. Two settings control it, and they are not the same thing as the runtime `OMDRC_SITE_DIR`:

Setting	Read by	Meaning
`OMDRC_SITE_DATA_DIRS`	CMake	semicolon-separated *search path* for `configs/<geo>` + `filters/<geo>`; first match wins
`OMDRC_SITE_ROOT`	the design scripts	the one checkout they read and write room data in (also `--site-root`)
`OMDRC_SITE_DIR`	`drc.sh` at runtime	the *installed* `$PREFIX/etc/open-media-drc`

Both default to the engine checkout, which is exactly the historical single-repository layout. Set the search path in `host.cmake`:

```
set(OMDRC_SITE_DATA_DIRS "${CMAKE_SOURCE_DIR};$ENV{HOME}/devel/omdrc-801N"
    CACHE STRING "Search path for configs/<geo> + filters/<geo>")
```

At configure time CMake prints which directory supplied each set, so the substitution is never silent:

```
-- core-drc: filter set search path
--     /home/giacomo/devel/open-media-drc (this checkout)
--     /home/giacomo/devel/omdrc-801N
--   120.blue (default) <- /home/giacomo/devel/omdrc-801N
--   flat <- /home/giacomo/devel/open-media-drc
```

A missing *extra* set is a warning and is skipped; a missing default `GEOMETRY` is a fatal error, because installing it would record a geometry in `omdrc.conf` with no configs behind it.

5.2 Geometry, design, variant

Three words that are easy to confuse:

- a **geometry** is a physical setup — speaker and listening position, e.g. `120.blue`. It is a directory under `configs/` and `filters/`.
- a **design** is one immutable filter revision inside a geometry, addressed as `@design-id` (e.g. `@rscreen-20260812`) and carrying a provenance manifest. The historical revision has the reserved id `default` and keeps the un-suffixed paths.
- a **variant** is the older mechanism: an arbitrary suffix appended to the config filename. It survives for compatibility, has no manifest, and is therefore always displayed as unverified. New work should use designs.

5.3 The bundle

Everything belonging to one design lives in the site repository under the geometry:

```
filters/<geometry>/
  provenance/<design>.json          the manifest (the commit marker)
  provenance/<design>.source.json   the build recipe (development input)
```

```
  analysis/<design>.json            precomputed response traces
  source/<design>/                  verbatim copies of the REW exports used
    measurement-L.txt  measurement-R.txt  measurement-L+R.txt
    filter-L.txt  filter-R.txt  filter-L.wav  filter-R.wav
    corrected-*-independent.txt     optional REW cross-check exports
  <rate>/{L,R}.raw                  runtime coefficients (design `default`)
  <rate>/@<design>/{L,R}.raw        runtime coefficients (immutable design)
configs/<geometry>/
  brutefir-<rate>.conf.in           template; @REPO_DIR@ -> $SITE_DIR at install
  brutefir-<rate>@<design>.conf.in
```

Sources are copied in under stable logical role names — the original REW filenames survive as manifest metadata. The copy is deliberate: a deployment must never depend on a mutable sibling checkout or on an absolute path that will not exist on the playback machine.

5.4 What is hashed

The manifest records, for every source export, RAW file, config template and the analysis file: the logical role, relative path, byte size, format, sample rate/count where applicable, and SHA-256. Around that it records the source repository URL and exact commit, the annotated tag name and immutable tag object id, the declaration's Git blob and SHA-256, parsed REW header metadata (measurement name, date, notes, smoothing, frequency step, timing reference, REW version), the derivation commands and tool versions (deployment script, Python, NumPy, SoX and the resampling flags), the TXT-versus-WAV validation results, per-channel headroom with safety margin and required attenuation, and the exact rate-to-config mapping with the expected BruteFIR format and attenuation.

The `bundle_id` on top is the SHA-256 of a canonical identity object containing a hash of the complete source-provenance block, every source artifact hash, the runtime config and RAW hashes and settings, and the analysis hash. Editing any provenance value the UI displays therefore invalidates the bundle rather than quietly changing a label. An annotated tag makes the source state an immutable named Git object; signing that tag additionally establishes authorship.

5.5 The scripts

All of them are offline. None starts REW, and none opens a `.mdat` project except the optional auditor below.

Script	Role
`filter_design_suggest.py`	read-only discovery: from a source checkout, finds the newest `.mdat` by mtime, locates its sibling `<stem>.txts`, ranks compatible L/R exports and prints a candidate declaration command. A suggestion, never an attestation

Script	Role
`declare_filter_design.py`	writes the source-side `design.json`: binds each semantic role to an exact file, records SHA-256 values and parsed TXT headers. Run in the source repository, then commit and tag
`new_filter_design.py`	consumes a committed declaration *at an annotated tag*: verifies the tag object, the commit, a clean tree and every input hash, then drives a dry run or a publication
`deploy_filter.py`	the engine underneath: regenerates every declared rate in a temporary directory, validates TXT against WAV, checks optional corrected exports, computes headroom, bakes and reads back each config, computes the analysis traces, and writes the bundle
`verify_filter_bundle.py`	read-only re-verification of committed bundles: bundle id, source copies, analysis dependencies, configs, exact RAW hashes and headroom. `--no-next` for CMake and CI
`filter_workflow_next.py`	shared operator handoff — prints the exact commit/install/select/verify commands, and names a working directory per step when the site data lives in its own repository
`headroom_calc.py`	minimum `attenuation:` per pair from the worst-case FFT gain plus a safety margin; also reports what each config currently specifies
`rew_mdat_audit.py`	optional archival evidence: audits selected REW project traces via the REW API, comparing TXT responses numerically and final WAV impulses sample-by-sample against the project. Not a deployment dependency
`REW2raw.sh`, `REW2raw-all-rates.sh`	the low-level SoX conversion underneath, usable directly for experiments; they produce no provenance bundle

5.6 The workflow

```
# 1. In the source repository: suggest, review, declare, tag.
python3 scripts/declare_filter_design.py --suggest-from-source-root
↪  ../DRC/DRC-120.blue
python3 scripts/declare_filter_design.py --geometry 120.blue --design-id
↪  rscreen-20260812 ...
git -C ../DRC/DRC-120.blue add ... && git -C ../DRC/DRC-120.blue commit -m
↪  'Declare ...'
git -C ../DRC/DRC-120.blue tag -a 120.blue-rscreen-20260812 -m 'room correction
↪  ...'
```

```
# 2. In the engine repository: audit as a dry run, read every PASS line.
export OMDRC_SITE_ROOT=~/devel/omdrc-801N
python3 scripts/new_filter_design.py --source-root ../DRC/DRC-120.blue \
        --source-ref 120.blue-rscreen-20260812 --declaration <path>

# 3. Publish only after reading the audit.
python3 scripts/new_filter_design.py ... --write

# 4. Re-verify independently, then commit in the SITE repository.
python3 scripts/verify_filter_bundle.py --all --require-sources
git -C ~/devel/omdrc-801N add -- configs/120.blue filters/120.blue
git -C ~/devel/omdrc-801N commit -m 'Deploy verified filter design: ...' && git
↪  -C ~/devel/omdrc-801N push
```

An annotated tag is required. `--allow-commit-ref` exists as an explicit lower-assurance exception and is recorded as such.

Publication is a transaction. Without `--write` every check runs in temporary storage and nothing is touched; the dry run also reports which runtime files *would* change. With `--write` the order is fixed: source copies first, then the analysis, then the runtime RAW pairs, and the manifest **last**. The manifest is the commit marker — readers ignore an incomplete deployment until it exists and verifies every preceding hash. Replacing bytes that already exist additionally requires `--replace-runtime`, so an accidental overwrite of a deployed design cannot happen silently.

5.7 Verification at install time

`cmake/core-drc.cmake` runs `verify_filter_bundle.py --require-sources` over every manifest of a geometry *before* installing it, and fails the configure step if a bundle does not verify. It also rejects any `@design` config that has no same-named manifest. A broken bundle therefore stops the build rather than producing an installed system that looks authoritative.

The install copies the manifests and analysis JSON beside the RAWs, and deliberately excludes `rew/`, `source/` and the `.source.json` recipes: their hashes and parsed metadata are already embedded in the installed manifest, so the playback machine needs none of the development material.

5.8 Verification at runtime

The response page never trusts the geometry name or the rate. For every request `omdrc-ctrl`:

1. finds the `.conf` of the **running** BruteFIR process from its argv;
2. parses the coefficient blocks and hashes the exact `.raw` files named there;
3. requires exactly one manifest matching that `(relative path, SHA-256, format, attenuation, rate)` tuple;
4. verifies the analysis file's own hash and that its recorded input hashes equal the manifest's source artifact hashes;
5. computes the displayed active-filter curve from the bytes just read, applying the configured attenuation, since that attenuation is part of the audible transfer function.

Only then are the stored room measurements released, with the green banner carrying the annotated tag, tag-object SHA and source commit; the details panel adds the declaration hash, bundle id and the active L/R RAW hashes.

Anything else is **mismatch** (red): no manifest matches the active bytes, a hash differs, the config attenuation or format differs, or an analysis dependency fails. The page may still show a live FFT of the active coefficients as a diagnostic, but it must not present stored measurements as if they belonged to it. A legacy variant, having no manifest, always lands here.

A/B switching obeys the same rule. After `drc.sh` returns, the server re-reads the running process, parses the config actually in use, hashes its RAWs, and reports the new selector as verified only if that runtime identity matches one manifest. A design that starts but fails this check is an assurance failure, not a successful switch.

5.9 What deliberately cannot go green

- A pair whose bytes do not reproduce from the recorded source exports. It must be re-exported and redeployed as a new bundle.
- Any selector without a manifest, whatever its audio quality.
- A design whose configured attenuation is below the computed requirement: the base `120.blue` pairs peak at about +1.28 dB and need 2.3 dB including the 1 dB margin, and are configured at 3.0 dB. A design whose filters never exceed unity gain legitimately requires 0.0 dB.

One gap is worth naming explicitly: the manifest pins the config *template*, not the rendered `.conf` that BruteFIR loads, because `@REPO_DIR@` is only substituted at install time. Runtime verification closes this for everything that matters — coefficients, format, attenuation and rate are all re-checked against the bytes in use — but a post-install hand-edit to a rendered config's routing or device settings is outside the chain.

Chapter 6

Tools

6.1 omdrc-ctrl — the web control panel

A Flask app serving a dark, touch-friendly control panel to any browser on the LAN (default `0.0.0.0:9090`). Originally a replacement for KDE Connect's feedback-less "Run command" plugin. Everything is driven by a plain INI file, `commands.conf`; no command is hard-coded.

Widget types — each `[section]` of `commands.conf` is one command:

- **READ** — runs a shell command, shows its output next to a label, optionally auto-refreshing (`refresh = N` seconds). A READ widget with `details_root` gains a dynamic **Details** button whenever `{details_root}/{output}/README.md` exists — so each filter configuration can carry its own documentation page with images.
- **WRITE** — a labelled button firing a command; green on success, red on failure, optional confirmation dialog (`confirm = yes`).
- **LINK** — opens a URL in a new tab.

Built-in monitoring panels (always present, below the command cards):

- **MPD panel** — the audio-health centrepiece for a headless server: daemon state, playback state and song, the stream MPD reports (rate/bits/ channels), the **DAC feed** (ALSA `hw_params` read from `/proc/asound` on Linux; the `virtual_oss` rate on FreeBSD), the BruteFIR rate, a green/red **SAMPLE RATE MATCH / RESAMPLING** comparison, and a plain-language **bit-perfect verdict**: *Bit-perfect passthrough* (DRC off, all rates equal), *Full-resolution DRC, no resampling* (BruteFIR at native rate), or *Resampling active*.
- **Qobuz Connect panel** — current track via qobuzconnect2mpd, with restart button and colour-coded log viewer; plus a renderer switch (qobuzconnect2mpd vs upmpdcli — never both) driven per-OS (`systemctl --user` on Linux, `sudo service ... onestart/onestop` on FreeBSD).
- **BruteFIR CPU** — per-process CPU for every brutefir instance (matched by `argv[0]`, because on Linux brutefir renames its `comm`).
- **Audio Devices** — `/dev/sndstat` on FreeBSD with `fmt 0x...` bitfields decoded to `AFMT_*`/`PCM_CAP_*` labels; collapsible.
- **Advanced** (FreeBSD only) — `sysctl dev.pcm.0` and `sysctl hw.usb.uaudio` diagnostics.
- **Top CPU** — processes above a configurable threshold.
- **Debug card** — the glitch-detection switch (section 6.3).

DRC filter response page — charts the *live* filters loaded by the running BruteFIR: magnitude (dB), delay-compensated wrapped phase, and residual group delay, computed on demand by FFT (NumPy) from the active config's `L.raw`/`R.raw`. Chart.js is vendored, so the page works offline. The filter files are only ever read, never modified.

Live spectrum analyzer — an optional card fed by a secondary MPD `fifo` output (`OMDRC Spectrum`, raw S32_LE stereo copy of the stream):

- Started/stopped from the page; the MPD FIFO output is enabled only while at least one browser is streaming (Server-Sent Events; multiple clients share one capture thread), force-disabled at startup for crash recovery.
- 24 logarithmic bands from 31.5 Hz, 10 Hz refresh, Music (16384-point) vs Precision (65536-point) FFT windows, VU bars or needles measured over a ~50 ms trailing window, one Floor slider driving graphs and meters.
- **DRC sync**: the tap is pre-DRC, so while BruteFIR runs the display is held back by the measured BruteFIR path delay (filter group delay + partition latency, cached and recomputed only when the active config changes) so it matches the audible sound.

Install: CMake. `cmake .. && sudo cmake --install .` installs system-wide (systemd unit on Linux, rc.d script on FreeBSD); `-DUSER_INSTALL=ON` (Linux only) installs to `~/.local` with a `systemd --user` unit — configure it *as the target user, without sudo*, and `loginctl enable-linger` for headless boxes. On FreeBSD: `sysrc omdrcctrl_enable=YES && service omdrcctrl start`; the rc.d script uses `daemon(8)`, drops privileges via the standard rc.subr `${name}_user`, and keeps its pidfile in a subdirectory of `/var/run` created by `start_precmd` (plain `/var/run/*.pid` would be root-only).

Security: the server executes arbitrary shell commands from `commands.conf` as the service user — trusted LAN only, never a public interface.

6.2 Video: mpv playback + phone web remote

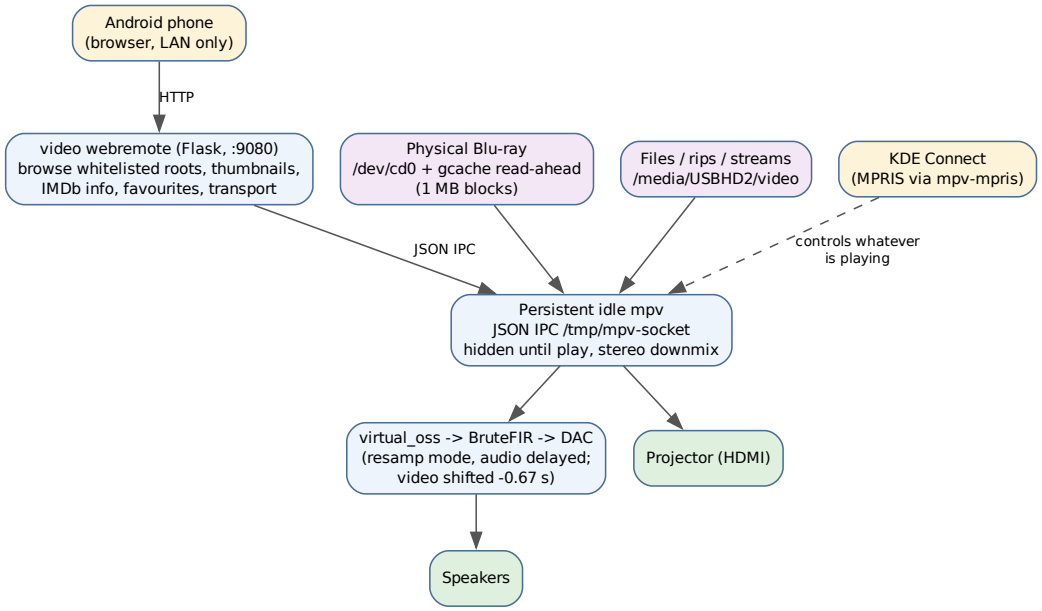

Figure 6.1: Video playback and control paths.

6.2.1 Playback launchers

- **`play-bluray.sh`** — physical Blu-ray discs. Kodi cannot read a physical BD on FreeBSD (raw `/dev/cd0` wants 2048-byte-aligned reads; the kernel cannot mount UDF 2.50), so mpv + libbluray read the raw device. Because the USB drive only sustains full speed in ~1 MB chunks and raw `cd0` has no kernel read-ahead, the script fronts the drive with a **GEOM cache** (`gcache create -b 1048576 -s 268435456 bd cd0` -> `/dev/cache/bd`), and probes `bd_list_titles` to play the *genuinely longest* title (mpv has no BD menu support; `e`/`E` cycle titles at runtime).
- **`play-media.sh`** — local files, playlists, and network/stream URLs (m3u8, yt-dlp sites); same DRC audio routing, no gcache.

Both source `lib/drc-audio.sh`, which ensures the chain is in **resamp mode** before playing — necessary because the direct DAC is bit-perfect (`dev.pcm.0.bitperfect=1`): a 48 kHz movie on a higher-clocked DAC would play ~2x fast. With DRC up, mpv plays to `oss//dev/dsp.play` and delays the **video** by `DRC_VIDEO_DELAY` (default **0.67 s**) to match the audio-path latency; subtitles ride with the picture automatically.

The 0.67 s is derived, not guessed (full derivation in `video/AV-SYNC-DELAY.md`): the FIR filter's impulse peak sits at sample 96000 of 524288 taps at 192 kHz = **0.500 s group delay** (the coefficient list *is* the impulse response; the peak is when a transient emerges), plus one BruteFIR partition (32768 samples at 192 kHz = **0.171 s**), plus a little `virtual_oss` buffering.

DVDs are simpler: they mount fine (UDF 1.x) and are low-bitrate, so `mpv dvd://`

`--dvd-device=/dev/cd0`; commercial discs need `libdvdcss`.

Remote control: the `mpv-mpris` package auto-loads into every mpv, so KDE Connect's Android *Media control* (and `playerctl`, Plasma widgets) drive whatever is playing — play/pause/seek/volume/metadata.

6.2.2 The web remote (`video/webremote/`)

KDE Connect controls what is *already playing*; the web remote is what *starts* a title. A separate Flask app (port 9080, LAN-only, rc.d service `omdrcvideo`) serving a phone UI to:

- **Browse** the whitelisted media roots (realpath containment — no `..` or symlink escape), with entries classified server-side: folder, Blu-ray rip (`BDMV/index.bdmv`), DVD rip (`VIDEO_TS`), or playable file. Grid (poster thumbnails) or compact list.
- **Thumbnails** via ffmpeg frame grabs, disk-cached by path+mtime, with a background prewarm thread and bounded concurrency.
- **IMDb info** — title deduced from the name and, with an OMDb API key, verified and enriched (year, director, cast, plot, rating).
- **Play** on a **persistent idle mpv** over its JSON IPC socket (`/tmp/mpv-socket`) — hidden until something plays, DRC audio configured once at startup, `audio-channels=stereo` so 5.1/7.1 sources downmix. Blu-ray rips get the longest-title probe; a **Play Blu-ray disc** button reuses the gcache lifecycle for physical discs, loading into the same mpv.
- **Transport** — seek, +-10/30 s, play/pause, mute, stop, audio and subtitle track menus; **favourites** pinned to the main page.

The idle mpv is autostarted by the KDE/Plasma session, from `~/.config/autostart/mpv-idle.desktop` (linked to the installed entry by `make user-install`); a `git pull` + `service omdrcvideo restart` is the whole update path.

6.3 Glitch detection

Implemented for FreeBSD (on Linux the same scripts run but the FreeBSD-specific sources are simply quiet). One global switch — `glitch-debug.sh on|off|status|analyze|usbtap|tail|clear` — also exposed as the Debug card in omdrc-ctrl.

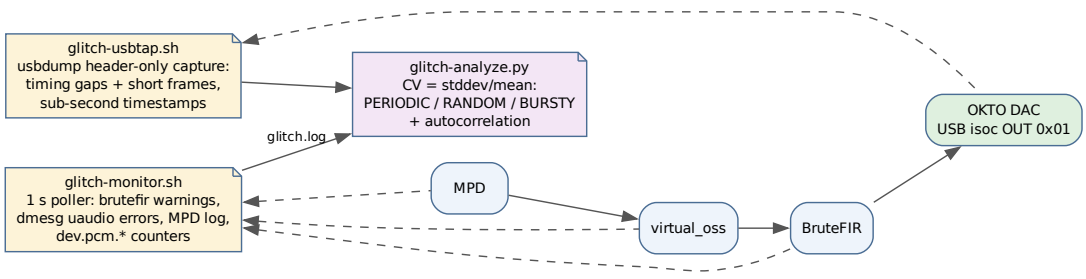

Figure 6.2: The glitch-detection layers and where each taps the chain.

- **`glitch-monitor.sh`** (always-on, lightweight): polls every second and logs new anomalies from

four sources — BruteFIR warnings (missed real-time deadlines), kernel `uaudio`/USB errors, the MPD log, and any increasing `dev.pcm.*` under/over/err/xrun counter — into a unified `glitch.log`.

- `glitch-usbtap.sh` (definitive, heavier, CLI-only): taps the OKTO's isochronous OUT endpoint 0x01 with `usbdump`, downstream of every software stage. Header-only analysis scales to multi-minute captures; it flags **timing gaps** (> 2.5x the nominal ~4 ms interval) and **short frames** (SLEN < 0.5x nominal), while the constant +-few-samples feedback wobble of asynchronous USB is counted separately and never flagged. Blind spot: a full-length block of zeros (silence insertion) needs payload inspection — use `verify-bitperfect.sh` for that.
- `glitch-analyze.py`: classifies inter-event intervals per stage by the coefficient of variation — CV ~ 0 **PERIODIC** (a buffer/clock cycle), CV ~ 1 **RANDOM/Poisson** (CPU/scheduling), CV > 1.5 **BURSTY** (something waking up) — plus autocorrelation and correlation against DRC rate switches in `drc.log`.

6.4 Bit-perfect verification

`scripts/verify-bitperfect.sh` *proves* the DAC receives bytes unchanged. Two levels:

1. **Structural** (kernel-certified): with `hw.snd.verbose=2`, `/dev/sndstat` must show the play channel `BITPERFECT` with the feeder graph exactly `{userland} -> feeder_root -> {hardware}` — any `feeder_rate`/`feeder_volume`/format node means the kernel is altering bytes. Preconditions: `dev.pcm.0.bitperfect=1`, `dev.pcm.0.play.vchans=0`.
2. **Empirical wire tap** (gold standard): play a deterministic test signal (near-silent ~-90 dBFS per-sample counter in the low 16 bits, distinct L/R — maximally sensitive to truncation, dither, volume, resampling, channel swap) while capturing the USB isochronous OUT endpoint 0x01 with `usbdump`; decode, align, and byte-compare. The embedded OSS writer aborts loudly if the kernel coerces format/channels/rate. Verified on this host at 44.1/48/88.2 kHz: hundreds of kB contiguous identical bytes, and on the live MPD both direct (**BIT-PERFECT**) and through `virtual_oss` (**VALUE-EXACT**, 0 slips).

The subtle part is clock domains: a producer must be **flow-controlled by the sink's clock** (blocked writes). MPD is; a free-running test writer is not, and drifts. `virtual_oss` itself is bit-transparent with a flow-controlled producer. The one caveat: BruteFIR bridges the loopback's software clock to the DAC crystal without resampling, so an inaudible one-sample slip occurs every several minutes on the DRC path — values are never altered. The DRC path is *intentionally* not byte-equal (that is the correction); to test its plumbing, use a unit-impulse filter with attenuation 0.

The test asset (`tests/`) is a 44.1 kHz S32 WAV whose PCM payload is byte-identical to the reference raw — MPD cannot play headerless raw.

6.5 scripts/ — helper tools

Script	Purpose
`REW2raw.sh`	REW WAV -> brutefir raw FLOAT64_LE at a target rate, with the theoretically correct `Fs_source/Fs_target` coefficient scale (no peak normalisation)

Script	Purpose
`REW2raw-all-rates.sh`	Batch: `L.raw/R.raw/sox.txt` for every numeric rate directory under a filter root; prompts before overwriting unless `-y`
`headroom_calc.py`	Minimum `attenuation:` per config from worst-case FFT gain + safety margin
`filter_design_suggest.py`	Read-only discovery of a candidate declaration command from a source checkout
`declare_filter_design.py`	Writes the source-side role declaration with hashes and parsed TXT headers
`new_filter_design.py`	Publishes a design from a committed declaration at an annotated tag
`deploy_filter.py`	The offline audit/build engine underneath: rates, validation, analysis, manifest
`verify_filter_bundle.py`	Read-only re-verification of committed bundles (used by CMake and CI)
`filter_workflow_next.py`	Shared operator handoff printed by the commands above
`rew_mdat_audit.py`	Optional archival audit of REW project traces against exports
`verify-bitperfect.sh`	The bit-perfect proof tool (above); sources: built-in writer or `mpd:OUTPUT`; taps: `usb` or `loop:/dev/dsp.X`
`systemd-user-install.sh`	Legacy: link + enable a `systemd --user` drc.service (Linux)

Chapter 7

FreeBSD peculiarities

A summary of everything FreeBSD-specific, in one place.

7.1 Naming and packages

- **MPD is `musicpd`**: package `audio/musicpd`, binary `musicpd`, service `musicpd`, client `musicpc` (vs `mpd`/`mpc` on Linux). omdrc-ctrl detects and uses the right client automatically.
- Services are rc.d scripts under `/usr/local/etc/rc.d/`, enabled with `sysrc <name>_enable=YES`, run manually with `service <name> onestart/onestop`. Hotplug is devd, not udev.

7.2 OSS instead of ALSA; virtual_oss and cuse

FreeBSD's native audio API is OSS. The loopback is `virtual_oss` (userland, base system) creating cuse character devices: `/dev/dsp.play` (MPD writes) and `/dev/dsp.loop` (BruteFIR reads, synchronized `-L` mode). The `cuse` kernel module must be loaded (`kld_list` or `etc/rc.d` glue). BruteFIR's OSS I/O is built in (no ALSA needed); the fork's OSS fixes matter here.

Key sysctls for the bit-perfect direct path:

```
dev.pcm.0.bitperfect=1     # first opener's format becomes the hardware format
dev.pcm.0.play.vchans=0    # no virtual-channel mixer/resampler
```

These also mean `/dev/dsp0` is **single-open**: exactly one client at a time (BruteFIR when DRC is on; MPD's direct output or a browser otherwise).

7.3 Known FreeBSD issues and their status

- **OKTO 44.1 kHz-family flicker** (bug #295933): the DAC continuously drops and re-acquires USB streaming lock on 44.1/88.2/176.4/352.8 kHz, while the 48 kHz family is stable and Linux plays everything fine. Root cause: the device has one UAC2 Clock Source **shared** between playback and capture, and `uaudio(4)` lets the vestigial capture side reprogram it to its 48 kHz default under the active playback. Fixed by the shared-clock patch (Appendix 8.1).
- **Rate-change cold-open silence**: on the first open after *any* rate change the DAC shows the right rate and streams healthy USB but routes no audio; a second open fixes it. Stock

`uaudio` programs the clock *after* selecting the streaming alt-setting (Linux does the opposite). Worked around by `drc.sh` priming; properly fixed by the clock-before-alt patch.

- **virtual_oss livelock** ("155% CPU", frozen chain): a `SNDCTL_DSP_SETTRIGGER` on a read-only fd could strand the synchronized engine in a wait that nothing wakes. Fixed by the settrigger patch (Appendix 8.2).
- **cuse teardown wedge** (bug #296291): stopping virtual_oss could leave it unkillable in `D<E` state, pinning `cuse.ko`, reboot required — a kernel refcount leak in `cuse_client_open()`'s `is_closing` error path (regression from commit `634e578ac7b0`, in 15.1). Kernel fix written (Appendix 8.2).
- **musicpd 100% CPU on HTTP streams**: a libcurl leftover-fd spin — curl registers an event fd into MPD's I/O loop and never removes it, so the loop burns a core in `poll()`. **Audio is unaffected**; MPD upstream has triaged it third-party (issues #2244/#2229); the right venue is libcurl.
- **One wire format per attach**: `uaudio` fixes s32le at attach and pads 16-bit content to 32 bits on the wire, so the DAC panel shows 24 bits on 16-bit tracks. Bit-perfect (zero-padding is lossless), just unlike Linux's per-stream alt switching. Knobs: `hw.usb.uaudio.default_bits`/`default_rate`.

7.4 Video-related FreeBSD constraints

- Physical Blu-ray: no kernel UDF 2.50 mount; raw `/dev/cd0` needs sector-aligned reads and has no read-ahead — hence mpv + libbluray + gcache (section 6.2). Kodi's internal player cannot do it.
- Kodi's OSS sink does not enumerate cuse userspace devices at all — the in-tree Kodi patch fixes that (Appendix 8.3).

7.5 Toward a real FreeBSD port

The path from the run-from-repo model to an official `audio/open-media-drc` port is planned in four phases — upstreaming the patches, splitting engine from site data, the port itself, and submission. The full plan is Appendix 9.

Chapter 8

Appendix A — FreeBSD patches in detail

Local fixes kept in-tree while the official FreeBSD fixes are pending. All `/usr/src` patches apply with `-p1` against `releng/15.1`.

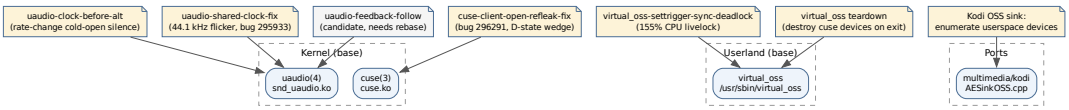

Figure 8.1: The patched layers at a glance.

> **Upgrade caveat (applies to every kernel/userland patch below):** `freebsd-update` or `make installkernel` overwrites patched binaries with stock ones. After any OS/kernel update, re-apply the patches and rebuild. A `.ko` is ABI-specific to its kernel — never keep prebuilt binaries, always rebuild from the patches.

8.1 uaudio(4) patches (`freebsd-uaudio-patch/`)

Target device: OKTO RESEARCH DAC8 STEREO (USB `0x152a:0x88c5`, Thesycon UAC2 firmware, USB High-Speed). Two patches are applied in order on top of stock `releng/15.1`; a third is an unbuilt candidate.

8.1.1 1. `uaudio-clock-before-alt.c.patch` — rate-change cold-open silence

Problem. On the first open after *any* sample-rate change the DAC opens the stream, shows the correct rate, streams healthy USB (feedback present, no underruns) — and routes silence. A second open at the same rate fixes it, which is why `drc.sh` had to prime (and why users had to "run drc.sh several times").

Root cause. Stock `uaudio_configure_msg_sub()` starts a stream as: `SET_INTERFACE` to the streaming alt-setting (arming the device), *then* `SET_CUR` the sample rate on the UAC2 Clock Source (possibly a crystal switch, yanked under the armed interface), then start transfers. Linux does the

opposite: park at alt 0, program the clock idle, then select the alt. The Thesycon firmware latches its stream configuration at `SET_INTERFACE` time, so FreeBSD's order arms the stream against the *old* clock. This also explains why the open/close prime works: every close parks at alt 0.

The fix. For UAC2 devices in the `CHAN_OP_START` path: park the streaming interface at alt 0; program the clock while idle (legal — the UAC2 clock lives on the AudioControl interface); on a genuine rate change sleep `hw.usb.uaudio.clock_settle_ms` (default 100 ms, runtime-tunable, clamped to 2000) for crystal relock; then `SET_INTERFACE` and start. UAC1 devices are untouched (their rate control lives on the streaming endpoint).

Status. Built + installed since 2026-07-06; listening reports say different-rate tracks now lock first try. Replaces the host-side `DAC_PRIME_CYCLES` prime for all clients (drc.sh, MPD-direct, browsers). If a crossing is ever silent, raise `sysctl hw.usb.uaudio.clock_settle_ms`.

8.1.2 2. `uaudio-shared-clock-fix.c.patch` — the 44.1 kHz flicker (bug #295933)

Problem. Continuous drop/re-acquire of USB streaming lock on the 44.1 kHz rate family; the 48 kHz family is stable; Linux plays everything.

Root cause. The DAC exposes **one UAC2 Clock Source shared between playback and capture**. On async playback, `uaudio` auto-starts the record channel purely as a jitter-information source — at its own nominal rate (48 kHz default), whose `SET_CUR` clobbers the shared clock out from under active 44.1 kHz-family playback.

The fix — three cooperating, device-agnostic changes:

a. **Rate-align the jitter record stream**: before the auto-started record channel starts, set its alt to the one matching the playback rate, so the rec-side `SET_CUR` becomes a same-value no-op *and* the rec channel's framing stays consistent (valid jitter feedback instead of catastrophically wrong).
b. **Shared-clock guard**: before any UAC2 `SET_CUR` to a clock shared by both directions, skip it if the other direction is running at a different rate — the first active stream owns the clock.
c. **Always submit the explicit-feedback SYNC transfer**, so `dev.pcm.0.feedback_rate` stays live as a diagnostic (drc.sh's chain-sanity signal) even with capture present.

A guard-only version was audited and rejected: without (a), the rec channel expects 48 kHz framing while the device correctly delivers 44.1 kHz, the jitter estimate pins at its negative clamp, and the play callback strips samples continuously — worse than the flicker. This fix **replaces** the retired VID/PID-gated capture-disable workaround; `pcm0 (play/rec)` in sndstat is the *expected* state again.

Status. Applied to `/usr/src` 2026-07-07, builds `-Werror`-clean alone and on top of patch 1; the combined module was pending install + listening test at the time of writing. Upstream: bug #295933 / PR 2323.

8.1.3 3. `uaudio-feedback-follow.c.patch` — candidate (unbuilt)

Sketch to make playback follow the device's reported feedback rate smoothly (Linux-style), targeting an occasional tick. Touches the same sync-callback region as fix (c) above, so it **needs rebasing** before any use.

8.1.4 Applying and building

```
cd /usr/src
patch -p1 < uaudio-clock-before-alt.c.patch
patch -p1 < uaudio-shared-clock-fix.c.patch   # applies with offsets; also on pure
↪  stock
patch -p1 < Makefile.patch                     # adds -DUSB_DEBUG

cd /usr/src/sys/modules/sound/driver/uaudio
make clean && make
```

Install the module (back up stock first, refresh the backup after every OS update so the revert path matches the running ABI):

```
OBJ=/usr/obj/usr/src/amd64.amd64/sys/modules/sound/driver/uaudio/snd_uaudio.ko
sudo cp -f /boot/kernel/snd_uaudio.ko /boot/kernel/snd_uaudio.ko.orig
sudo service musicpd stop                  # release /dev/dsp0
sudo cp -f "$OBJ" /boot/kernel/snd_uaudio.ko
sudo kldunload snd_uaudio                  # devd auto-reloads on attach
UG=$(usbconfig | awk '/DAC8STEREO/{print $1}' | tr -d ':')
sudo usbconfig -d "$UG" reset              # clean re-enumeration
sudo sysctl -f /etc/sysctl.conf            # restore buffer_ms baseline
sudo service musicpd start
```

Verify: `grep pcm0 /dev/sndstat` shows `(play/rec)`; `sysctl hw.usb.uaudio.clock_settle_ms` exists; `sysctl dev.pcm.0.feedback_rate` tracks the playback rate. Then run the listening matrix (both crystal directions, same-family changes, MPD-direct mixed-rate queue, browsers) — machine signals cannot verify these fixes; only listening counts. Revert with the `.orig` copy + kldunload/kldload.

8.2 virtual_oss / cuse patches (`freebsd-virtual-oss-patch/`)

Two related bug clusters, both root-caused live on this box.

8.2.1 1. Runtime livelock: `virtual_oss-settrigger-sync-deadlock.patch`

Symptom. Minutes after a chain (re)start, playback freezes: MPD stops advancing, BruteFIR starves, `virtual_oss` burns 150–200% CPU, both clients stuck in `cuse-cli` waits. Stopping anything from this state walks into the teardown minefield below.

Root cause (captured with procstat + gdb on the live process): a pair of userland bugs in `usr.sbin/virtual_oss`:

1. `SNDCTL_DSP_SETTRIGGER` ignores the fd's open mode: BruteFIR's OSS layer triggers both directions with one call (harmless on kernel pcm), flipping `tx_enabled = 1` on the read-only `/dev/dsp.loop` fd that can never write.
2. The synchronized-loopback engine wait loops check `tx_enabled` **once, outside the wait**, and the trigger/halt ioctl paths never wake the engine — so once the engine parks waiting for play data from a client that will never write, nothing ever re-evaluates the premise. The arming window is microseconds wide against a 200 ms block cadence — hence intermittent.

The fix (four changes, upstreamable): SETTRIGGER honours the open mode (`fflags`); trigger/halt ioctls `atomic_wakeup()` the engine; the sync wait loops re-check `tx_enabled/rx_enabled`; and a new `tx_written` latch so the engine never sleeps waiting for a client that has never written.

8.2.2 2. Teardown deadlock: userland device destroy + kernel refleak

Symptom. Stopping `virtual_oss` intermittently wedges it forever in `D<E` / `MWCHAN W` (SIGKILL-immune), pins `cuse.ko` (kldunload hangs too), a new virtual_oss cannot recreate the devices — reboot required. Every DRC rate change was a reboot risk.

Two layers:

- **Userland**: `virtual_oss` never destroyed its cuse devices on exit, so the kernel's `cuse_server_free()` busy-waited in `pause("W", hz)` for client refs that were never released. Patches `virtual_oss-teardown-int.h.patch` + `virtual_oss-teardown-main.c.patch` keep each `cuse_dev_create()` handle and call `cuse_dev_destroy()` on all DSP/WAV/loopback devices before exit. (Upstream committed an equivalent as `0bd5ef6b4363`.)
- **Kernel** (the real fix — bug #296291): a regression from commit `634e578ac7b0` (Nov 2025, in 15.1). In `cuse_client_open()`, the `is_closing` / `si_drv1==NULL` error path returns with `pcs->refs` incremented, the client left linked in `hcli`, and the destructor never registered — **every open racing into the teardown window leaks one server ref**, and `cuse_server_free()` waits forever. Proven live with a diagnostic `cuse.ko` + dtrace + kgdb (refs frozen at 4, three leaked clients on `dsp.loop`). Fix: `cuse-client-open-refleak-fix.patch` calls `cuse_client_free(pcc)` on both error paths; committed on branch `fix/cuse-client-open-refleak-296291`, compile-tested, `Fixes: 634e578ac7b0`, `PR: 296291`. The userland destroy mitigates the normal exit but does nothing for `kill -9` — the kernel leak stays latent without this.
- A diagnostic-only `cuse-teardown-diag.c.patch` logs the stuck refcount every ~5 s from the `pause("W")` loop (pure printf; remove once the kernel fix lands).

8.2.3 Applying and building

```
cd /usr/src
patch -p1 < virtual_oss-settrigger-sync-deadlock.patch
patch -p1 < virtual_oss-teardown-int.h.patch
patch -p1 < virtual_oss-teardown-main.c.patch
patch -p1 < cuse-teardown-diag.c.patch        # optional diagnostic

# userland:
cd /usr/src/usr.sbin/virtual_oss && make && sudo make install

# kernel module (diagnostic or refleak fix):
cd /usr/src/sys/modules/cuse && make && sudo make install
```

Reboot first if a wedged virtual_oss is pinning `cuse.ko` — a new module cannot load and the devices cannot be recreated until then. Validation: churn `drc.sh <rate>` / `off` cycles ~20x; `virtual_oss` must stay near-idle CPU and always exit cleanly (no `D<E` in `ps -o pid,stat,mwchan`, `cuse.ko` unloadable).

8.3 Kodi OSS sink patch (`kodi-virtual-oss-patch/`)

Problem. Kodi's audio settings only ever offered the hardware DAC. `CAESinkOSS::EnumerateDevicesEx()` counts kernel PCM cards via `SNDCTL_SYSINFO`; cuse userspace devices are not kernel cards, so `/dev/dsp.play` was never listed — and the settings filler silently *resets* any configured value it cannot match against the enumerated list, so hand-editing `guisettings.xml` never stuck.

The fix. After the kernel-card loop, parse the *"Installed devices from userspace:"* section of `/dev/sndstat` (the only place cuse devices are advertised) and add each node, probed exactly like a kernel card (`SNDCTL_ENGINEINFO` for formats/channels/rates, non-blocking open, graceful fallbacks so a busy device is still listed, dedup against kernel cards). Generic — any userspace OSS device is listed.

Result (verified on `multimedia/kodi` 22.0a3): *"dsp.play virtual_oss device"* appears in Settings > System > Audio, selecting it persists, and Kodi feeds the DRC chain. `virtual_oss` must be running when Kodi initialises audio.

Apply via the ports tree:

```
sudo cp patch-xbmc_cores_AudioEngine_Sinks_AESinkOSS.cpp \
        /usr/ports/multimedia/kodi/files/
cd /usr/ports/multimedia/kodi
make config && make patch && make build
sudo make deinstall && sudo make install   # same version -> reinstall is a no-op
strings /usr/local/lib/kodi/kodi.bin | grep -c "from userspace"   # must print 1
```

Upstreaming: to the FreeBSD port (`files/` patch, PR to the maintainer) and/or Kodi upstream (`AESinkOSS.cpp` PR; pre-empt the "why parse sndstat text" question — cuse devices are unreachable via the mixer ioctls).

Chapter 9

Appendix B — The FreeBSD port plan

Status: **plan only, nothing applied.** Linux packaging is explicitly out of scope for now (one OS at a time). Source: `doc/FREEBSD-PORT-PLAN.md`.

9.1 Why the repo cannot be ported as-is

A FreeBSD port installs *identical, immutable* files on every machine, under hier(7) paths, from a versioned release tarball. The run-from-repo model violates that on every axis — deliberately, because it optimizes for a zero-config personal appliance:

1. **Files are rendered per-host**: `install.sh` bakes `config.env` values (`@AUDIO_USER@`, `@AUDIO_HOME@`, `@REPO_DIR@`) into the live files; a package must install the same bytes everywhere and configure at runtime.
2. **The tree is written at runtime**: `drc.sh` keeps `last_arg`, `last_power`, `drc.log` beside itself; `pkg check -s` flags modified packaged files — state must live in `/var/db/`.
3. **Room-specific data is mixed with software**: `configs/120.blue`, `filters/*` (~50 MB) are personal measurement products; a port must ship neutral defaults. `OMDRC_SITE_DATA_DIRS` / `OMDRC_SITE_ROOT` now resolve these in a separate checkout beside the engine.
4. **rc.d scripts shadow other ports**: `etc/rc.d/musicpd` and `upmpdcli` would replace scripts owned by `audio/musicpd` / `net/upmpdcli`; the stock scripts' rc.conf knobs must be used instead.
5. **Dependency on a personal BruteFIR fork**: `RUN_DEPENDS` must resolve to ports.
6. **Kernel/userland patches**: a port cannot patch the base system (uaudio, cuse) and should not carry patches for another port (virtual_oss) — these must land upstream first.
7. **Missing packaging basics**: no LICENSE, no tagged releases (the `v*` tag series this manual belongs to is the first step), and the tarball would ship debugging journals and kernel patches.

Run-from-repo does not have to die: the repo becomes a normal upstream that *also* supports `make install PREFIX=... DESTDIR=...`; run-from-repo stays the development/appliance mode and the port is a thin consumer of tagged releases.

9.2 Phase 0 — Upstream the out-of-tree pieces (prerequisite, in flight)

- **uaudio patches** (shared-clock fix, clock-before-alt) to FreeBSD base — in progress: bug 295933 / PR 2323.
- **cuse refleak fix** to base — in progress: bug 296291.
- **virtual_oss SETTRIGGER deadlock patch** to upstream `hselasky/virtual_oss`, so `audio/virtual_oss` inherits it.
- **BruteFIR fork** — options in order of preference: (a) upstream the delta to Anders Torger (upstream is dormant — unlikely); (b) submit the delta as `files/` patches to the existing `audio/brutefir` port (viable if the delta stays small and FreeBSD-relevant); (c) release the fork as its own project + port (`audio/brutefir-omdrc`) — most work, only if (b) is refused.
- **kodi-virtual-oss-patch**: not shippable by this port — upstream to Kodi or to `multimedia/kodi`'s `files/`, or drop from the tarball.

9.3 Phase 1 — Make the repo package-friendly

Worth doing even if the port never happens; it makes the repo usable by anyone, not just this box/room.

- **1.1 Engine / site-data split.** Engine (shipped): drc.sh, drc-status.sh, omdrc-ctrl, video/webremote, browser-nodrc, own rc.d/devd glue, sample MPD/upmpdcli snippets, docs. Site data (not shipped): the `configs/` and `filters/` measurement products, room README sections, measurement plots — moved to `examples/rooms/` or a separate private overlay checked out beside the engine (`OMDRC_SITE_DIR`).
- **1.2 FLAT default filters** (first — small and independent): ship a `configs/flat/` geometry as default, one conf per rate, using BruteFIR's built-in identity coefficient `filename: "dirac pulse"` with `attenuation: 0.0` — no binary filters needed, and `verify-bitperfect.sh` can pass through the flat chain as a plumbing self-test. Keep `filter_length` and I/O identical to the room configs so swapping in real filters is a filename change.
- **1.3 Runtime configuration instead of render-time baking**: drc.sh and friends read a config at runtime (`$OMDRC_CONF -> ${PREFIX}/etc/open-media-drc/omdrc.conf -> <script-dir>/config.env`, the last keeping run-from-repo unchanged); rc.d defaults flip to installed paths via the port's `SUB_FILES`; brutefir confs rendered on the fly to a state-dir tempfile so packaged confs are host-neutral.
- **1.4 State out of the tree**: `last_arg`, `last_power`, `drc.log` to `/var/db/omdrc/` in installed mode (beside the script as fallback); `/tmp/brutefir.out`, `/tmp/virtual_oss.pid` to the same state/run dir.
- **1.5 Install target** (`make install` with `DESTDIR`/`PREFIX`): scripts to `${PREFIX}/libexec/omdrc/` + a `${PREFIX}/bin/omdrc` wrapper; `omdrc.conf.sample` + flat configs to `${PREFIX}/etc/open-media-drc/`; devd conf; MPD/upmpdcli snippets to `share/examples/`; omdrc-ctrl to `share/omdrc-ctrl/`; docs to `share/doc/open-media-drc/`. rc.d scripts for our own services are installed by the port via `USE_RC_SUBR`.
- **1.6 Housekeeping**: pick a LICENSE (BSD-2-Clause fits the ecosystem); tagged releases whose tarballs exclude `freebsd-*-patch/`, investigation journals and site data (`git archive` + `export-ignore`); split the README into a user quickstart vs `doc/DEVELOPMENT.md`.

9.4 Phase 2 — The port itself

`audio/open-media-drc` (working name): `USE_GITHUB=yes`, tagged `DISTVERSION`, `NO_BUILD` for the shell core, `USES=python:run shebangfix` for omdrc-ctrl. `RUN_DEPENDS`: brutefir (per the Phase 0 decision), virtual_oss, musicpd, sox/soxr. `OPTIONS_DEFINE`: `CTRL` (web UI: py-flask/Markdown/numpy), `VIDEO` (webremote: mpv), `UPNP` (upmpdcli). `USE_RC_SUBR` for `drc_usb_audio omdrcctrl omdrcvideo` — **not** musicpd or upmpdcli; a pkg-message documents the rc.conf lines pointing the stock scripts at our configs. `@sample` entries for every config. Validate with `portlint -AC`, `portclippy`, `poudriere testport`, and `pkg check -s` after a service run (catches leftover in-tree writes).

9.5 Phase 3 — Submission and maintenance

Submit as a Bugzilla PR (Ports & Packages), `MAINTAINER= delleceste@gmail.com`. Niche integration ports are accepted when clean and maintained — the bar is quality, not popularity. Expect review rounds on rc script style, sample handling, and the brutefir dependency; having the Phase 0 fixes merged upstream is the strongest argument the stack works on stock FreeBSD. Ongoing: bump the port per release, watch fallout from musicpd/upmpdcli/virtual_oss updates.

9.6 Recommended order of value

1. **Phase 0 upstreaming** — benefits every FreeBSD USB-audio user, already in flight, and a hard prerequisite anyway.
2. **Phase 1.2 (flat filters)** — small, immediate, makes the repo usable by others today even without a port.
3. **Rest of Phase 1** — worthwhile for the repo's own health regardless.
4. **Phases 2–3** — only once the BruteFIR question is settled and there is evidence of an audience; a port is a maintenance promise.

Chapter 10

Appendix C — Bit-perfect test assets and cross-OS comparison

The Tools chapter (*Bit-perfect verification*) covers the single-host proof. This appendix documents the test assets and the cross-OS procedure that proves the Linux and FreeBSD boxes send the DAC the *very same bytes*.

10.1 Test assets (`tests/`)

Two deterministic, near-silent (~ -90 dBFS) signals; every sample is uniquely determined, so any truncation, dither, volume change, resampling or channel swap shows up immediately.

- **Short asset (committed)** — `bitperfect-test-44100-s32-stereo.wav`: S32_LE, 2 ch, 44100 Hz, 100000 frames (~2.27 s). Per-sample counter in the low 16 bits, `L = i & 0xFFFF`, `R = (i*40503) & 0xFFFF`. Its PCM payload is byte-identical to the reference `.raw` (the WAV is the same bytes plus a 44-byte header — MPD cannot play headerless raw).
- **Cross-OS asset (generated, not committed)** — `bitperfect-test-44100-s32-stereo-30s.wav`: S32_LE, 2 ch, 44100 Hz, 1323000 frames (30 s), 10.6 MB. Here *every* `(L,R)` pair is unique over the whole file (`R = (i*40503 + (i >> 16)) & 0xFFFF` folds the block index in, breaking the 65536-frame period), so capture alignment is unambiguous at any length. It is too large to commit; regenerate it byte-identically on any OS:

```
python3 tests/gen-bitperfect-wav.py \
    tests/bitperfect-test-44100-s32-stereo-30s.wav
# sha256 88d365eeaccb1fa830bb1a2726b0f29bb545885824351080e0c5b4cbc9602348
```

10.2 Other rates and sample widths

The generator takes `--rate`, `--bits` (16/24/32) and `--frames` (= seconds x rate), so any format the DAC advertises can be tested. Both tap scripts read rate and width from the WAV header — feeding them a different file is the whole configuration:

```
python3 tests/gen-bitperfect-wav.py --rate 192000 --bits 24 --frames 5760000 \
    tests/bitperfect-test-192000-s24-stereo-30s.wav
```

```
./scripts/bitperfect-tap-linux.sh tests/bitperfect-test-192000-s24-stereo-30s.wav
```

The sample *values* are the same counter at every width (they never exceed `0xFFFF`), so only the container changes — which means a 16/24-bit asset additionally exercises the **lossless promotion to the 32-bit USB wire container** (`<<8` for 24-bit, `<<16` for 16-bit) that any bit-perfect player must perform for a DAC accepting only 32-bit containers. A 16/24-bit input therefore does not change the playback path at all: `prep` promotes first and the player always emits S32_LE.

Two consequences: **each format has its own sha256** (a 24-bit file stores 3 bytes per sample, so it is a different file), and a cross-OS comparison must use the **same width on both machines** — differently shifted wire values never byte-match. The report's `ref bytes` hash is that of the promoted stream, so it too differs per width at the same rate.

Rate	Bits	Frames	Size	sha256 (first 16)
44100	32	1323000 (30 s)	10584044	`88d365eeaccb1fa8`
44100	24	1323000 (30 s)	7938044	`e2702c119606cdf8`
192000	24	5760000 (30 s)	34560044	`58dd87f3560334fb`
192000	24	1920000 (10 s)	11520044	`01317af6523ec67f`
96000	24	960000 (10 s)	5760044	`b572faabdee3b623`

Any other combination is equally valid: the generator prints the sha256 of whatever it writes — generate once, note the hash, match it on the other machine (full hashes in `tests/README.md`). `.gitignore` excludes the generated assets by name (the 44100/32-bit and 44100/24-bit 30 s files and the 192000/24-bit 10 s one), so add any further WAV you keep, or drop it under `bp-results/` (ignored except for `*.txt`).

10.3 Verification status

Both taps are executed and passing — the FreeBSD side is no longer a written-but-unrun script:

Host	Runs	Result
Linux (DacMagic 100, kernel 7.1.5-arch1)	44100/32-bit x 30 s, 44100/24-bit x 30 s, 192000/24-bit x 10 s	all **BIT-PERFECT**, 0 truncated events, 0 usbmon drops
FreeBSD 15.1-RELEASE (same DAC, `usbus0` devaddr 2)	same three	all **BIT-PERFECT** (exit 0); the 192 kHz run confirms the DAC clock followed (`dev.pcm.0.feedback_rate` = 191994) and costs ~24 s wall clock

`bitperfect-compare.py` reports **MATCH** across the two hosts for the 44100/32-bit asset; the 24-bit pairs have no committed Linux counterpart yet, so those stand as local per-host proofs. The comparator itself has been exercised on every path (wav/wav, wav/txt, txt/txt, refusal of raw/txt) plus a deliberately bit-flipped payload, which it reports as MISMATCH at the exact offset.

The FreeBSD tap's first run exposed a real defect — a capture truncated by ~17 ms because `usbdump` discards its unflushed buffer on exit. The fix is a 500 ms **silence pad**: the player plays `ref.raw` plus half a second of zeros, while the verdict still compares the unpadded reference, so the loss lands in silence nothing depends on. The pad is removed by *arithmetic* (take `len(ref)` bytes from the alignment point), never by silence detection — the test signal is itself near-silent, so a zero-seeking trim would eat real payload.

10.4 Cross-OS byte comparison

Each OS taps its own USB isochronous OUT endpoint while playing the (locally regenerated) common WAV, then the reports are compared. Only the tiny `bp-results/*.txt` reports are committed — they carry the tap payload's length and sha256, which proves byte-identity without moving the 10 MB streams through git.

```
# Linux box:
./scripts/bitperfect-tap-linux.sh \
    tests/bitperfect-test-44100-s32-stereo-30s.wav
git add bp-results/*-linux.txt && git commit && git push

# FreeBSD box (free the DAC first: ./drc.sh off):
./scripts/bitperfect-tap-freebsd.sh \
    tests/bitperfect-test-44100-s32-stereo-30s.wav
git add bp-results/*-freebsd.txt && git commit && git push

# then on either box:
git pull
./scripts/bitperfect-compare.py \
    bp-results/bitperfect-test-44100-s32-stereo-30s-linux.txt \
    bp-results/bitperfect-test-44100-s32-stereo-30s-freebsd.txt
```

Identical length and sha256 on both reports proves the two operating systems deliver bit-identical audio to the DAC. Step-by-step commands and the mismatch-forensics path are in `scripts/README.md` and `doc/BIT-PERFECT-VERIFICATION.md` (*Cross-OS comparison*).

A single run already proves the local path. Each tap compares its capture against the reference derived from the input file *on that machine* and exits 0 on **BIT-PERFECT** — a complete file -> USB proof by itself. `bitperfect-compare.py` is a separate, optional step answering the further question of whether two hosts agree.

What the report records. Each `PREFIX.txt` names and hashes every stage (`input file`, `ref bytes`, `wire raw`, `tap wav`, `verdict`), so a run is auditable from the ~600-byte report alone. Only three of those are reproducible: `input file`, `ref bytes` and `tap wav`. **`wire raw` is not** — it is the untrimmed capture, so its length varies between otherwise identical runs (a few packets more or fewer recorded before the tap stops), and Linux and FreeBSD captures of the same input legitimately differ (10584816 vs 10772776 bytes at 44100/32-bit). It is provenance only; the field the comparator uses is `tap wav`. Note also that `PREFIX.wav` equals the input WAV only for a **32-bit** input: for 16/24-bit it carries the promoted 32-bit container, so it is longer and differently valued — the invariant that always holds is the `tap wav` payload hash, not the file hash.

What surrounds the audio. The capture is always longer than the reference, and everything

outside is measurably all-zero: a head of stream-priming zeros — **exactly 16 ms at both 44100 and 192000 Hz**, a fixed-duration buffer prime rather than a timing accident — and a tail of the 500 ms pad plus ~19 ms the kernel keeps transmitting after the writer closes. On a **BIT-PERFECT** verdict both capture boundaries necessarily fell outside the audio; when they fall inside, the tool names it (`HEAD LOST` at the start, `INCOMPLETE` at the end) rather than hiding it. One qualification: "inaudible" describes the sample *values*. Opening or closing an isochronous stream, and any rate change around it, can still produce an audible artifact from the DAC's analogue side (mute relay, PLL relock — see `OKTO-DAC8-FreeBSD-44k1-flicker.md`); that comes from stream start/stop, not from the zeros.

Start with the canonical 44100 Hz asset on FreeBSD. The FreeBSD tap decodes `usbdump -vv` *text*, so parsing cost scales with the capture: 30 s at 44100 Hz is ~10.5 MB of payload arriving as tens of MB of hex-dump text (fine), while 30 s at 192 kHz is ~46 MB of payload as several hundred MB of text — slow, and it stresses the pcap capture too. Prove the path at 44100 first, then shorten the high-rate run (`--frames 1920000` = 10 s at 192 kHz). The Linux side reads usbmon's binary interface and has no such limit.

Chapter 11

Appendix D — Source document index

This manual is generated from the repository's source files — the Markdown docs below plus the CMake build for the Installation chapter. For the full detail behind each section:

Topic	Source document
Chain overview, install, drc.sh, hotplug	`README.md`
Install / build (CMake superproject, host values)	`CMakeLists.txt`, `host.cmake.sample`
Build modules: DRC engine + site data	`cmake/core-drc.cmake`
Build modules: DAC-hotplug + brutefir services	`cmake/hotplug.cmake`
Build modules: MPD + upmpdcli renderers	`cmake/renderers.cmake`
Build modules: runtime dependency audit	`cmake/dependencies.cmake`
Build modules: per-user setup (`make user-install`)	`cmake/user-install.sh.in`
Web-UI subproject builds	`omdrc-ctrl/CMakeLists.txt`, `video/webremote/CMakeLists.txt`
Filter/config layout, drc.sh modes, agent rules	`FILTERS_AND_DRC.md`
Filter provenance, hashes, verification, the design scripts	`doc/FILTER_PROVENANCE_AND_RESPONSE.md`
Site-data split (`OMDRC_SITE_DATA_DIRS` / `OMDRC_SITE_ROOT`)	`scripts/README.md`, `host.cmake.sample`, `cmake/core-drc.cmake`
Helper scripts	`scripts/README.md`, `README.md`
Web control panel	`omdrc-ctrl/README.md`
Spectrum analyzer	`omdrc-ctrl/SPECTRUM_ANALYZER.md`
Video playback + Blu-ray	`video/README.md`
A/V sync delay derivation	`video/AV-SYNC-DELAY.md`
Web remote (install/API)	`video/webremote/README.md`
Web remote (design)	`video/webremote/ARCHITECTURE.md`
Glitch detection	`doc/GLITCH-DETECTION.md`
Bit-perfect verification	`doc/BIT-PERFECT-VERIFICATION.md`
Test signal	`tests/README.md`
FreeBSD port plan	`doc/FREEBSD-PORT-PLAN.md`
uaudio patches (index + install)	`freebsd-uaudio-patch/README.md`
44.1 kHz flicker analysis	`freebsd-uaudio-patch/FreeBSD-uaudio-shared-clock-bug`
Shared-clock fix design	`freebsd-uaudio-patch/uaudio-shared-clock-fix.md`

Topic	Source document
Clock-before-alt analysis	freebsd-uaudio-patch/uaudio-clock-before-alt.md
Feedback-follow audit	freebsd-uaudio-patch/uaudio-feedback-follow.md
virtual_oss patches (index)	freebsd-virtual-oss-patch/README.md
SETTRIGGER livelock root cause	freebsd-virtual-oss-patch/ROOTCAUSE-settrigger-sync-e
cuse refleak root cause	freebsd-virtual-oss-patch/ROOTCAUSE-cuse_client_open-
cuse teardown bug report	VIRTUAL_OSS_CUSE_DEADLOCK.md
Cold-open silence audit	OKTO-DAC8-silent-first-open.md
44.1 kHz flicker observations	OKTO-DAC8-FreeBSD-44k1-flicker.md
MPD/curl CPU spin	MPD-CURL-CPU-SPIN-FreeBSD.md
Kodi OSS sink patch	kodi-virtual-oss-patch/README.md
VBA vs all-pass comparison	doc/xtras/FVBA.vs.ALLPASS.md

11.1 Keeping this manual up to date

This manual is a **synthesis** of the source documents above, not a transclusion: rebuilding the PDF does *not* pull in changes to the source `.md` files automatically. When a source document changes:

1. Update the corresponding section of the master document, `doc/pdf/open-media-drc-manual.md` (the table above is the section-to-source mapping).
2. If the chain topology changed, adjust the graphviz sources in `doc/pdf/diagrams/*.dot`.
3. Re-run `doc/pdf/build-pdf.sh` (requires pandoc, pdflatex, graphviz) to regenerate `doc/open-media-drc-manual.pdf`.

Editing constraints for the master document (pdflatex): keep it ASCII — no box-drawing characters or Unicode arrows/symbols; diagrams are added as `![caption](build/<name>.pdf){width=NN%}`. Full details in `doc/pdf/README.md`.